

第 3 章 最简单的C 程序设计

——顺序程序设计

有了前两章的基础，现在可以开始由浅入深地学习C 语言程序设计了。

为了能编写出C 语言程序，必须具备以下的知识和能力：

(1) 要有正确的解题思路，即学会设计算法，否则无从入手。

(2) 掌握C 语言的语法，知道怎样使用C 语言所提供的功能编写出一个完整的、正确的程序。也就是在设计好算法之后，能用C 语言正确表示此算法。

(3) 在写算法和编写程序时，要采用结构化程序设计方法，编写出结构化的程序。

算法的种类很多，不可能等到把所有算法都学透以后再来学习编程序。C 语言的语法规则很多，很烦琐，孤立地学习语法不但枯燥乏味，而且即使倒背如流，也不一定能写出一个好的程序，必须找到一种有效的学习方法。

本书的做法是：以程序设计为主线，把算法和语法紧密结合起来，引导读者由易及难地学会编写C 程序。对于简单的程序，算法比较简单，程序中涉及的语法现象也比较简单（一般只用到简单的变量、简单的输出格式）。对于比较复杂的算法，程序中用到的语法现象也比较复杂（例如要使用数组、指针和结构体等）。

本章先从简单的程序开始，介绍简单的算法，同时介绍最基本的语法现象，使读者具有编写简单的程序的能力。在此基础上，逐步介绍复杂一些的程序，介绍比较复杂的算法，同时介绍较深入的语法现象，把算法与语法有机地结合起来，由浅入深，由简单到复杂，使读者很自然地、循序渐进地学会编写程序。

3.1 顺序程序设计举例

【例3.1】有人用温度计测量出用华氏法表示的温度（如64° F），今要求把它转换为以摄氏法表示的温度（如17.8℃）。

解题思路：这个问题的算法很简单，关键在于找到二者间的转换公式。根据物理学知

识，知道以下转换公式：

$$c = \frac{5}{9}(f - 32)$$

其中f 代表华氏温度，
c 代表摄氏温度。据

此可以用N-S 图表示算 法，见图3. 1。

算法由3个步骤组成，这是一个简单的顺序结构。

输入f 的值
$c=\frac{5}{9}(f-32)$
输出c的值

图 3. 1

编写程序：有 了N-S 图，很容易用C 语言表示，写出求此问题的C 程序。

```
#include<stdio.h>
int main()
```

```
float f,c;
f=64.0;
c=(5.0/9)*(f-32);
printf("f=%f\nc=%f\n",f,c);
return 0;
```

//定义f和c 为单精度浮点型变量
//指定 f的值
//利用公式计算c 的值
//输出c 的值

运行结果：

```
f=64.000000
c=17.777778
```

读者应能看懂这个简单的程序。

【例3.2】计算存款利息。有1000元，想存一年。有3种方法可选：(1)活期，年利率为r1;(2) 一年期定期，年利率为r2;(3) 存两次半年定期，年利率为r3。请分别计算出一年后按3种方法所得到的本息和。

解题思路：关键是确定计算本息和的公式。从数学知识可知，若存款额为p0，则：
活期存款一年后本息和为 $p1=p0(1+r1)$ 。
一年期定期存款，一年后本息和为 $p2=p0(1+r2)$ 。

两次半年定期存款，一年后本息和为 $p3=p0\left(1+\frac{r3}{2}\right)\left(1+\frac{r3}{2}\right)$ 。

画出N-S 流程图，见图3.2。

编写程序：按 照N-S 图所表示的算法，很容易写出C 程序。

```
#include<stdio.h>
int main()
{float      p0=1000,r1=0.0036,r2=0.0225,r3=0.0198,p1,p2,p3;

    p1=p0*(1+r1);

    p2=p0*(1+r2);

    p3=p0*(1+r3/2)*(1+r3/2);
    printf("p1=%f\np2=%f\np3=%f\n",p1,p2,p3);
    return 0;
```

//定义变量
//计算活期本息和
//计算一年定期本息和
//计算存两次半年定期的本息和
//输出结果

输入p0, r1, r2, r3的值
计算p1=p0(1+r1)
计算p2=p0(1+r2)
计算 $p3=p0\left(1+\frac{r3}{2}\right)\left(1+\frac{r3}{2}\right)$
输出p1, p2, p3

图 3.2

运行结果：

```
p1=1003.599976  
p2=1022.500000  
p3=1019.898010
```

第1行是活期存款一年后本息和，第2行是一年定期存款一年后本息和，第3行是两次半年定期存款一年后本息和。

 **程序分析：**第4行，在定义实型变量 p0,p1,p2,p3,r1,r2,r3 的同时，对变量p0, r1, r2, r3 赋予初值。

第8行，在输出 p1,p2 和 p3 的值之后，用\n 使输出换行。

注意：在 Visual C++6.0 系统中对以上两个程序进行编译时，会显示出“警告”信息。这是因为编译系统把所有实数都作为双精度数处理。因此提醒用户：把双精度常量转换成 float 型会造成精度损失。对这类“警告”，用户知道是怎么回事就可以了。承认此现实，让程序继续进行连接和运行，不影响运行结果。如果用 GCC 编译系统，则不会出现此“警告”信息。

3.2 数据的表现形式及其运算

有了以上写程序的基础，本节对程序中最基本的成分作必要的介绍。

说明：本节介绍的主要是 C 语言的一些语法规则，在编程序时会用到这些知识，因此不知道是不行的，所以本书作了简单的介绍。但是，不需要死记硬背，这样既枯燥又难以奏效，教师也不必在课堂中一一讲授。建议学习本节时采取“浏览”的方法，大致知道有这些因素就可以了，这样在遇到有关问题时就不会茫然。在后续的章节中，通过阅读程序和分析程序对这些内容会具体掌握的，必要时再回头查阅一下即可。

3.2.1 常量和变量

在计算机高级语言中，数据有两种表现形式：常量和变量。

1. 常量

在程序运行过程中，其值不能被改变的量称为常量。如例 3.1 程序中的 5, 9, 32 和例 3.2 程序中的 1000, 0.0036, 0.0225, 0.0198 是常量。数值常量就是数学中的常数。

常用的常量有以下几类：

(1) **整型常量。** 如 1000, 12345, 0, -345 等都是整型常量。

(2) **实型常量。** 有两种表示形式：

① 十进制小数形式，由数字和小数点组成。如 123.456, 0.345, -56.79, 0.0, 12.0 等。

② 指数形式，如 $12.34e3$ (代表 12.34×10^3)， $-346.87e-25$ (代表 -346.87×10^{-25})， $0.145E-25$ (代表 0.145×10^{-25}) 等。由于在计算机输入或输出时无法表示上角或下角，故规定以字母 e 或 E 代表以 10 为底的指数。但应注意：e 或 E 之前必须有数字，且 e 或 E 后面必须为整数。如不能写成 $e4$, $12e2.5$ 。

(3) **字符常量。** 有两种形式的字符常量：

① **普通字符**，用单撇号括起来的一个字符，如：'a', 'Z', '3', '?', '#'。不能写成 'ab' 或 '12'。请注意：单撇号只是界限符，字符常量只能是一个字符，不包括单撇号。'a' 和 'A' 是不同的字符常量。字符常量存储在计算机存储单元中时，并不是存储字符 (如 a, z, # 等) 本身，而是以其代码 (一般采用 ASCII 代码) 存储的，例如字符 'a' 的 ASCII 代码是 97，因此，在存储单元中存放的是 97 (以二进制形式存放)。ASCII 字符与代码对照表见附录 A①。

① C 语言并没有指定使用哪一种字符集，由各编译系统自行决定采用哪一种字符集。C 语言只是规定：基本字符集中的每个字符必须用一个字节表示；空字符也占一个字节，它的所有二进位都是0；对数字0~9字符的代码，后面一个数字的代码应比前一个数字的代码大1（如在ASCII 字符集中，数字' 2' 的代码是50，数字' 3' 的代码是51，后者比前者的代码大1，符合要求）。中小型计算机系统大都采用 ASCII 字符集，ASCII 是 American Standard Code for Information Interchange（美国标准信息交换代码）的缩写。

② **转义字符**，除了以上形式的字符常量外，C 语言还允许用一种特殊形式的字符常量，就是以字符“\”开头的字符序列。例如，前面已经遇到过的，在 `printf` 函数中的 `'\n'` 代表一个“换行”符。`t'` 代表将输出的位置跳到下一个 Tab 位置(制表位置)，一个 Tab 位置为8列。这是一种在屏幕上无法显示的“控制字符”，在程序中也无法用一个一般形式的字符来表示，只能采用这样的特殊形式来表示。

常用的以“\”开头的特殊字符见表3.1。

表 3.1 转义字符及其作用

转 义 字 符	字 符 值	输 出 结 果
<code>\'</code>	一个单撇号(')	输出单撇号字符'
<code>\"</code>	一个双撇号(")	输出双撇号字符"
<code>\?</code>	一个问号(?)	输出问号字符?
<code>\</code>	一个反斜线(\)	输出反斜线字符\
<code>\a</code>	警告(alert)	产生声音或视觉信号
<code>\b</code>	退格(backspace)	将光标当前位置后退一个字符
<code>\f</code>	换页(form feed)	将光标当前位置移到下一页的开头
<code>\n</code>	换行	将光标当前位置移到下一行的开头
<code>\r</code>	回车(carriage return)	将光标当前位置移到本行的开头
<code>\t</code>	水平制表符	将光标当前位置移到下一个Tab位置
<code>\v</code>	垂直制表符	将光标当前位置移到下一个垂直制表对齐点
<code>\o</code> 、 <code>\oo</code> 或 <code>\ooo</code> 其中o代表一个八进制数字	与该八进制码对应的 ASCII字符	与该八进制码对应的字符
<code>\xh[h...]</code> 其中h代表一个十六进制数字	与该十六进制码对应的 ASCII字符	与该十六进制码对应的字符

表3.1中列出的字符称为**转义字符**，意思是将“\”后面的字符转换成另外的意义。如“`\n`”中的“n”不代表字母n 而作为“换行”符。

表3.1中倒数第2行是一个以八进制数表示的字符，例如 `'\101'` 代表八进制数101的 ASCII 字符，即 `'A'` (八进制数101相当于十进制数65, 从附录A 可以看到ASCII 码(十进制数)为65的字符是大写字母'A') 。`'\012 '`代表八进制数12(即十进制数的10)的ASCII 码所对应的字符“换行”符。表3.1中倒数第1行是一个以十六进制数表示的 ASCII 字符，如 `'\x41'`代表十六进制数41的ASCII 字符，也是 `'A'` (十六进制数41相当于十进制数65)。用表3.1中的方法可以表示任何可显示的字母字符、数字字符、专用字符、图形字符和控制字

符。如 `'\033'` 或 `'x1B'` 代表ASCII 代码为27的字符,即ESC 控制符。`'\0'` 或 `'\000'` 是代表ASCII 码为0的控制字符,即“空操作”字符,它常用在字符串中。

(4) **字符串常量**。如 `"boy"`、`"123"` 等,用双撇号把若干个字符括起来,字符串常量是双撇号中的全部字符(但不包括双撇号本身)。注意不能错写成 `'CHINA'`、`'boy'`、`'123'`。单撇

号内只能包含一个字符，双撇号内可以包含一个字符串。

说明：从其字面形式上即可识别的常量称为“字面常量”或“直接常量”。字面常量是没有名字的不变量。

(5)符号常量。 用 `#define` 指令，指定用一个符号名称代表一个常量。如：

```
#define PI 3.1416 //注意行末没有分号
```

经过以上的指定后，本文件中从此行开始所有的PI 都代表3.1416。在对程序进行编译前，预处理器先对PI 进行处理，把所有 PI 全部替换为3.1416。这种用一个符号名代表一个常量的，称为**符号常量**。在预编译后，符号常量已全部变成字面常量(3.1416)。使用符号常量有以下好处。

①含义清楚。看程序时从PI 就可大致知道它代表圆周率。在定义符号常量名时应考虑“见名知义”。在一个规范的程序中不提倡使用很多的常数，如：`sum=15×30*23.5*43`，在检查程序时搞不清各个常数究竟代表什么：应尽量使用“见名知义”的变量名和符号常量。

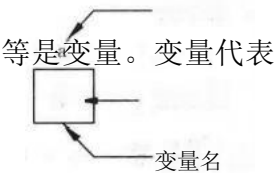
②在需要改变程序中多处用到的同一个常量时，能做到“一改全改”。例如在程序中多处用到某物品的价格，如果价格用一个常数30表示，则在价格调整为40时，就需要在程序中作多处修改，若用符号常量PRICE 代表价格，只须改动一处即可：

```
#define PRICE 40
```

注意：要区分符号常量和变量，不要把符号常量误认为变量。符号常量不占内存，只是一个临时符号，代表一个值，在预编译后这个符号就不存在了，故不能对符号常量赋新值。为与变量名相区别，习惯上符号常量用大写表示，如PI,PRICE 等。

2. 变量

如例3. 1程序中的c,f 和例3. 2程序中的p0,pl,p2,p3,rl,r2,r3 一个有名字的、具有特定属性的一个存储单元。它用来存放数据，也就是存放变量的值。在程序运行期间，变量的值是可以改变的。



变量必须先定义，后使用①。在定义时指定该变量的名字和类型。

一个变量应该有一个名字，以便被引用。请注意区分变量名和变量值这两个不同的概念，图3. 3中a 是变量名，3是变量a 的值，即存放在

图 3.3 变量值

变量a 的内存单元中的数据。变量名实际上是以一个名字代表的一个存储地址。在对程序编译连接时由编译系统给每一个变量名分配

存储单元

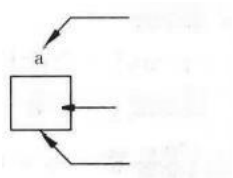
图 3.3

对应的内存地址。从变量中取值，实际上是通过变量名找到相应的内存地址，从该存储单元中读取数据。

3. 常变量

C99 允许使用**常变量**，方法是在定义变量时，前面加一个关键字const，如：

①定义变量的位置： 一般在函数开头的声明部分中定义变量，也可以在函数外定义变量(即外部变量、全局变量，见第7章)。 C 99允许在函数中的复合语句(用一对花括号包起来)中定义变量。



```
const int a=3;
```

定义 `a` 为一个整型变量, 指定其值为 3, 而且在变量存在期间其值不能改变。

常量与变量的异同是: 常量具有变量的基本属性: 有类型, 占存储单元, 只是不允许改变其值。可以说, 常量是有名字的不变量, 而常量是没有名字的不变量。有名字就便于在程序中被引用。

请思考: 常量与符号常量有什么不同?如:

```
#define Pi 3.1415926
const float pi=3.1415926;           //定义符号常量
                                     //定义常变量
```

符号常量 `Pi` 和常变量 `pi` 都代表 3.1415926, 在程序中都能使用。但二者性质不同: 定义符号常量用 `#define` 指令, 它是预编译指令, 它只是用符号常量代表一个字符串, 在预编译时仅进行字符替换, 在预编译后, 符号常量就不存在了(全替换成 3.1415926 了), 对符号常量的名字是不分配存储单元的。而常变量要占用存储单元, 有变量值, 只是该值不改变而已。从使用的角度看, 常变量具有符号常量的优点, 而且使用更方便。有了常变量以后, 可以不必多用符号常量。

 **说明:** 有些编译系统还未实现 C 99 的功能, 因此不能使用常变量。

4. 标识符

在计算机高级语言中, 用来对变量、符号常量名、函数、数组、类型等命名的有效字符序列统称为标识符(identifier)。简单地说, 标识符就是一个对象的名字。前面用到的变量名 `p1`, `p2`, `c`, `f`, 符号常量名 `PI`, `PRICE`, 函数名 `printf` 等都是标识符。

C 语言规定标识符只能由字母、数字和下画线 3 种字符组成, 且第 1 个字符必须为字母或下画线。下面列出的是合法的标识符, 可以作为变量名:

```
sum, average, _total, Class, day, month, Student_name, lotus_1_2_3, BASIC, li_ling。
```

下面是不合法的标识符和变量名:

```
M.D.John, Y123, #33, 3D64, a>b
```

注意: 编译系统认为大写字母和小写字母是两个不同的字符。因此, `sum` 和 `SUM` 是两个不同的变量名, 同样, `Class` 和 `class` 也是两个不同的变量名。一般而言, 变量名用小写字母表示, 与人们日常习惯一致, 以提高可读性。

3.2.2 数据类型

在例 3.1 和例 3.2 中可以看到: 在定义变量时需要指定变量的类型。如例 3.1 中变量 `f` 和 `c` 被定义为单精度(float)型。C 语言要求在定义所有的变量时都要指定变量的类型。常量也是区分类型的。

为什么在用计算机运算时要指定数据的类型呢?在数学中，数值是不分类型的，数值的运算是绝对准确的，例如：78与97之和为175, $1/3$ 的值是0.33333333...（循环小数）。数学是一门研究抽象问题的学科，数和数的运算都是抽象的。而在计算机中，数据是存放在存储单元中的，它是具体存在的。而且，存储单元是由有限的字节构成的，每一个存储单元中

存放数据的范围是有限的，不可能存放“无穷大”的数，也不能存放循环小数。例如用C 程序计算和输出1/3:

```
printf("%f",1.0/3.0);
```

得到的结果是0.333333,只能得到6位小数，而不是无穷位的小数。

注意：用计算机进行的计算不是抽象的理论值的计算，而是用工程的方法实现的计算，在许多情况下只能得到近似的结果。

所谓类型，就是对数据分配存储单元的安排，包括存储单元的长度(占多少字节)以及数据的存储形式。不同的类型分配不同的长度和存储形式。

C 语言允许使用的类型见图3.4, 图中有*的是C 99 所增加的。

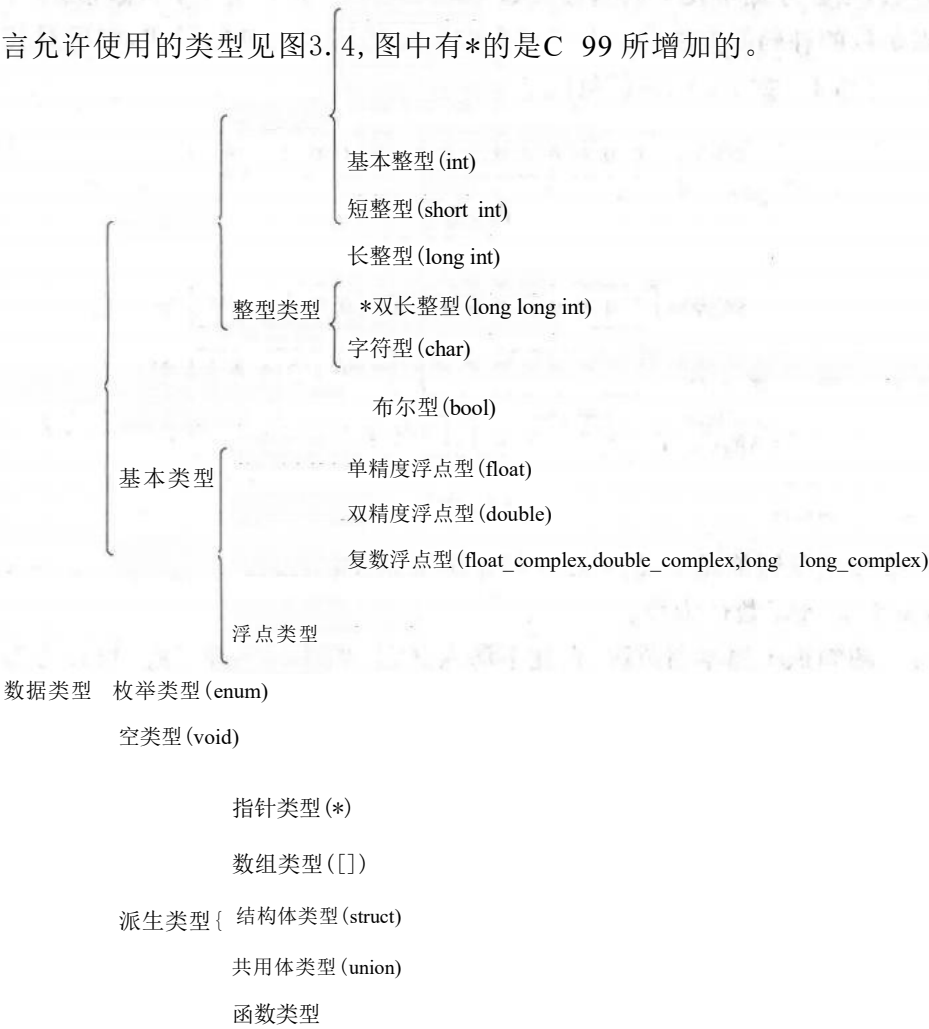
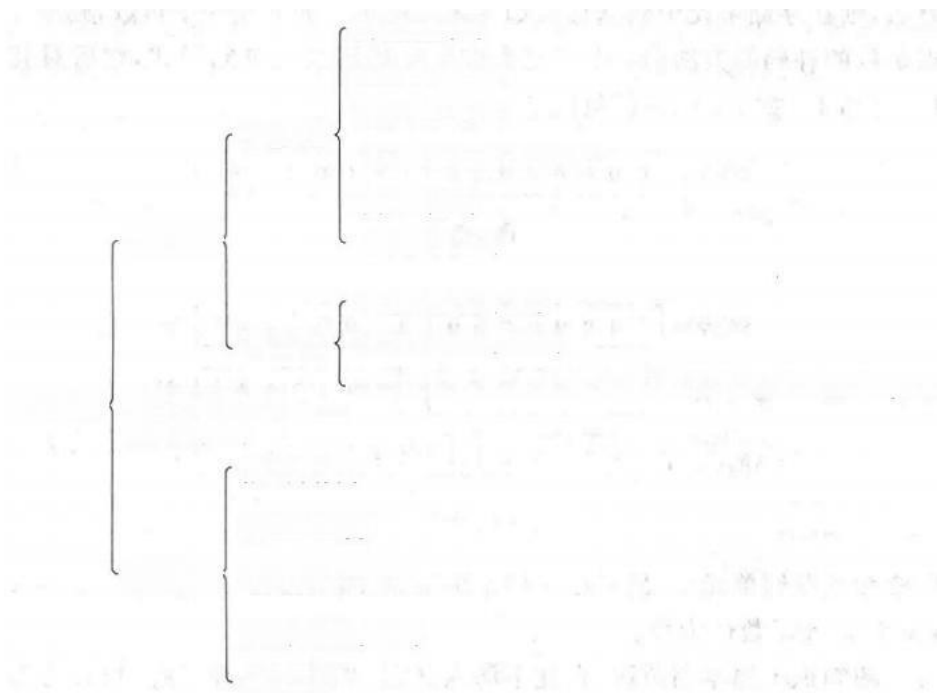


图 3.4

其中，基本类型(包括整型和浮点型)和枚举类型变量的值都是数值，统称为算术类型(arithmetic type)。算术类型和指针类型统称为纯量类型(scalar type),因为其变量的值是以数字来表示的。枚举类型是程序中用户定义的整数类型。数组类型和结构体类型统称为组合类型(aggregate type),共用体类型不属于组合类型，因为在同一时间内只有一个成员具有值。函数类型用来定义函数，描述一个函数的接口，包括函数返回值的数据类型和参数的类型。

不同类型的数据在内存中占用的存储单元长度是不同的，例如，Visual C++ 为 char 型 (字符型) 数据分配1个字节，为int 型 (基本整型) 数据分配4个字节，存储不同类型数据的方法也是不同的。

本书不孤立地、枯燥地叙述以上各种类型的规则，而是结合编程介绍怎样使用各种数据类型。本章及第4、5章介绍基本数据类型的应用，第6章介绍数组，第7章介绍函数，第8章介绍指针，第9章介绍结构体类型、共用体类型和枚举类型。



3.2.3 整型数据

1. 整型数据的分类

本节介绍最基本的整型类型。

(1) 基本整型(int 型)

编译系统分配给int 型数据2个字节或4个字节(由具体的C 编译系统自行决定)。如Turbo C 2.0 为每一个整型数据分配2个字节(16个二进位), 而Visual C++ 为每一个整型数据分配4个字节(32位)。在存储单元中的存储方式是: 用整数的补码(complement) 形式存放。一个正数的补码是此数的二进制形式, 如5的二进制形式是101, 如果用两个字节存放一个整数, 则在存储单元中数据形式如图3.5所示。如果是一个负数, 则应先求出负数的补码。求负数的补码的方法是: 先将此数的绝对值写成二进制形式, 然后对其所有二进位按位取反, 再加1。如-5的补码见图3.6。

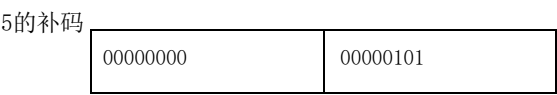


图 3.5

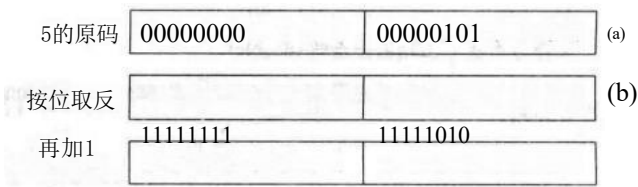


图 3.6

在存放整数的存储单元中, 最左面一位是用来表示符号的。如果该位为0, 表示数值为正; 如果该位为1, 表示数值为负。

有关补码的知识不属本书范围, 在此不深入介绍, 如需进一步了解, 可参考有关计算机原理的书籍。

说明: 如果给整型变量分配2个字节, 则存储单元中能存放的最大值为0111111111111111, 第1位为0代表正数, 后面15位为全1, 此数值是 $(2^{15}-1)$, 即十进制数32767。最小值为1000000000000000, 此数是-215即-32768。因此一个整型变量的值的范围是-32768~32767。超过此范围, 就出现数值的“溢出”, 输出的结果显然不正确。如果给整型变量分配4个字节(Visual C++), 其能容纳的数值范围为 $-2^{31} \sim (2^{31}-1)$, 即-2147483648~2147483647。

(2) 短整型(short int)

类型名为short int 或 short。如用Visual C++, 编译系统分配给int 数据4个字节, 短整型2个字节。存储方式与int 型相同。一个短整型变量的值的范围是-32768~32767。

(3) 长整型(long int)

类型名为 `long int` 或 `long`。Visual C++ 对一个 `long` 型数据分配4个字节(即32位), 因此`long int` 型变量的值的范围是 $-2^{31} \sim (2^{31}-1)$, 即 $-2147483648 \sim 2147483647$ 。

(4) 双长整型(long long int)

类型名为 long long int或 long long,一般分配8个字节。这是C99 新增的类型,但许多 C 编译系统尚未实现。

说明: C 标准没有具体规定各种类型数据所占用存储单元的长度,这是由各编译系统自行决定的。C 标准只要求 long 型数据长度不短于int 型, short 型不长于int 型。即

$$\text{sizeof}(\text{short})\leq\text{sizeof}(\text{int})\leq\text{sizeof}(\text{long})\leq\text{sizeof}(\text{long long})$$

sizeof 是测量类型或变量长度的运算符。在 Turbo C2.0中, int 型和 short 型数据都是2个字节(16位),而long 型数据是4个字节(32位)。在Visual C++中, short 数据的长度为2字节, int 数据的长度为4字节, long 数据的长度为4字节。通常的做法是:把 long 定为32位,把short 定为16位,而int 可以是16位,也可以是32位,由编译系统决定。读者应了解所用系统的规定。在将一个程序从 A 系统移到 B系统时,需要注意这个区别。例如,在 A 系统,整型数据占4个字节,程序中将整数50000赋给整型变量price 是合法的、可行的。但在B系统,整型数据占2个字节,将整数50000赋给整型变量price 就超过整型数据的范围,出现“溢出”。这时应当把 int 型变量改为 long 型,才能得到正确的结果。

2. 整型变量的符号属性

以上介绍的几种类型,变量值在存储单元中都是以补码形式存储的,存储单元中的第1个二进位制代表符号。整型变量的值的范围包括负数到正数(见表3. 2)。

表 3 . 2 整型数据常见的存储空间和值的范围 (Visual C++ 的 安 排)

类 型	字节数	取 值 范 围
int(基本整型)	4	-2147483648~2147483647, 即 $-2^{31}\sim(2^{31}-1)$
unsigned int(无符号基本整型)	4	0~4294967295, 即 $0\sim(2^{32}-1)$
short(短整型)	2	-32768~32767, 即 $-2^{15}\sim(2^{15}-1)$
unsigned short(无符号短整型)	2	0~65535, 即 $0\sim(2^{16}-1)$
long(长整型)	4	-2147483648~2147483647, 即 $-2^{31}\sim(2^{31}-1)$
unsigned long(无符号长整型)	4	0~4294967295, 即 $0\sim(2^{32}-1)$
long long(双长型)	8	-9223372036854775808~9223372036854775807 即 $-2^{63}\sim(2^{63}-1)$
unsigned long long (无符号双长整型)	8	0~18446744073709551615, 即 $0\sim(2^{64}-1)$

在实际应用中，有的数据的范围常常只有正值(如学号、年龄、库存量、存款额等)。为了充分利用变量的值的范围，可以将变量定义为“无符号”类型。可以在类型符号前面加上修饰符 `unsigned`，表示指定该变量是“无符号整数”类型。如果加上修饰符 `signed`，则是“有符号类型”。因此，在以上4种整型数据的基础上可以扩展为以下8种整型数据：

有符号基本整型	[signed]int
无符号基本整型	unsigned int
有符号短整型	[signed]short [int]
无符号短整型	unsigned short [int]
有符号长整型	[signed]long [int]
无符号长整型	unsigned long [int]
有符号双长整型*	[signed]long long [int]
无符号双长整型*	unsigned long long [int]

以上有“[]”的是C99 增加的，方括号表示其中的内容是可选的，既可以有，也可以没有。如果既未指定为signed 也未指定为unsigned 的，默认为“有符号类型”。如 signed int a 和 int a 等价。

有符号整型数据存储单元中最高位代表数值的符号(0为正，1为负)。如果指定 unsigned (为无符号)型，存储单元中全部二进位(b) 都用作存放数值本身，而没有符号。无符号型变量只能存放不带符号的整数，如123, 4687等，而不能存放负数，如-123, -3。由于左面最高位不再用来表示符号，而用来表示数值，因此无符号整型变量中可以存放的正数的范围比一般整型变量中正数的范围扩大一倍。如果在程序中定义a 和 b 两个短整型变量(占2个字节)，其中b 为无符号短整型：

```
short a; //a 为有符号短整型变量
unsigned short b; //b 为无符号短整型变量
```

则变量a 的数值范围为-32768~32767, 而变量b 的数值范围为0~65535。图3. 7 (a) 表示有符号整型变量 a 的最大值(32767), 图3. 7 (b) 表示无符号整型变量 b 的最大值(65535)。

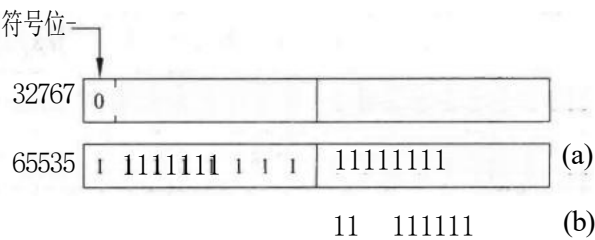


图 3. 7

- 说明：
- (1) 只有整型(包括字符型)数据可以加 signed 或 unsigned 修饰符，实型数据不能加。
 - (2) 对无符号整型数据用“%u”格式输出。%u 表示用无符号十进制数的格式输出。如：

```
unsigned short price=50;
printf("%u\n",price);
```

//定义 price 为无符号短整型变量

//指定用无符号十进制数的格式输出

在将一个变量定义为无符号整型后，不应向它赋予一个负值，否则会得到错误的结果。如：

```
unsigned short price=-1;
```

//不能把一个负整数存储在无符号变量中

```
printf("%d\n",price);
```

得到结果为65535。显然与原意不符。

请思考：这是为什么？

原因是：系统对-1先转换成补码形式，就是全部二进位都是1(见图3.8)，然后把它存入变量price中。由于price是无符号短整型变量，其左面第一位不代表符号，按“%d”格式输出，就是65535。

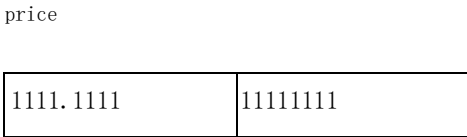


图 3.8

对以上补码的表示有初步了解即可，暂时可不细究。

说明：在程序中经常会对各种类型的数据进行操作，使用C语言编程时应当对数据在计算机内部的存储情况有一些基本的了解。否则，对运行时出现的问题会感到莫名其妙，无从分析。

3.2.4 字符型数据

由于字符是按其代码(整数)形式存储的，因此C99把字符型数据作为整数类型的一种。但是，字符型数据在使用上有自己的特点，因此把它单独列为一节来介绍。

1. 字符与字符代码

字符与字符代码并不是任意写一个字符，程序都能识别的。例如代表圆周率的 π 在程序中是不能识别的，只能使用系统的字符集中的字符，目前大多数系统采用ASCII字符集。各种字符集(包括ASCII字符集)的基本集都包括了127个字符。其中包括：

- 字母：大写英文字母A~Z, 小写英文字母a~z。
- 数字：0~9。
- 专门符号：29个，包括
! ” # & . ' () * + , - . / : ; < = > ? [\] ^ _ { | } ~
- 空格符：空格、水平制表符(tab)、垂直制表符、换行、换页(form feed)。
- 不能显示的字符：空(null) 字符(以' \0' 表示)、警告(以' \a' 表示)、退格(以' \b' 表示)、回车(以' \r' 表示)等。

详见附录A(ASCII 字符表)。这些字符用来写英文文章、材料或编程序基本够用了。前已说明，字符是以整数形式(字符的ASCII 代码)存放在内存单元中的。例如：大写字母' A' 的 ASCII 代码是十进制数65, 二进制形式为1000001。小写字母' a' 的 ASCII 代码是十进制数97, 二进制形式为1100001。数字字符' 1' 的ASCII 代码是十进制数49, 二进制形式为0110001。空格字符' ' 的 ASCII 代码是十进制数32, 二进制形式为0100000。

专用字符 '%' 的ASCII 代码是十进制数37, 二进制形式为0100101。

转义字符\n'的 ASCII 代码是十进制数10, 二进制形式为0001010。

可以看到, 以上字符的 ASCII 代码最多用7个二进位就可以表示。所有127个字符都

可以用7个二进位表示(ASCII 代码为127时, 二进制形式为1111111, 7位全1)。所以在C语言中, 指定用一个字节(8位)存储一个字符(所有系统都不例外)。此时, 字节中的第1位置为0。

如小写字母'a'在内存中的存储情况见图3.9('a'ASCII 代码是十进制数97,二进制数为01100001)。

注意：字符'1'和整数1是不同的概念。字符'1'只是代表一个形状为'1'的符号，在需要时按原样输出，在内存中以ASCII码形式存储，占1个字节，见图3.10(a)；而整数1是以整数存储方式(二进制补码方式)存储的，占2个或4个字节，见图3.10(b)。

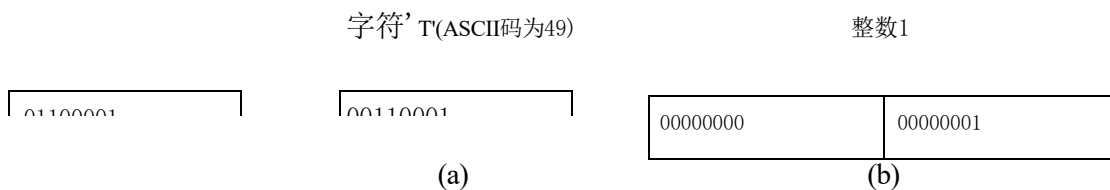


图 3.9

图 3.10

整数运算1+1等于整数2,而字符'1'+'1'并不等于整数2或字符'2'。

2. 字符变量

字符变量是用类型符char 定义字符变量。char 是英文character(字符)的缩写，见名即可知义。如：

```
char c='?';
```

定义c 为字符型变量并使初值为字符'?'。?'的ASCII 代码是63,系统把整数63赋给变量c。

c 是字符变量，实质上是一个字节的整型变量，由于它常用来存放字符，所以称为字符变量。可以把0~127之间的整数赋给一个字符变量。

在输出字符变量的值时，可以选择以十进制整数形式输出，或以字符形式输出。如：

```
printf("%d  %c\n",c,c);
```

输出结果是

63 ?

说明：用“%d”格式输出十进制整数63，用“%c”格式输出字符‘?’。

前面介绍了整型变量可以用signed 和 unsigned 修饰符表示符号属性。字符类型也属于整型，也可以用signed 和 unsigned 修饰符。

字符型数据的存储空间和值的范围见表3.3。

表3.3 字符型数据的存储空间和值的范围

类 型	字节数	取 值 范 围
-----	-----	---------

signed char(有符号字符型)	1	-128~127, 即 $-2^7 \sim (2^7-1)$
unsigned char(无符号字符型)	1	0~255, 即 $0 \sim (2^8-1)$

说明：在使用有符号字符型变量时，允许存储的值为 $-128 \sim 127$ ，但字符的代码不可能为负值，所以在存储字符时实际上只用到 $0 \sim 127$ 这一部分，其第1位都是0①。

3.2.5 浮点型数据

浮点型数据是用来表示具有小数点的实数的。为什么在C中把实数称为浮点数呢？在C语言中，实数是以指数形式存放在存储单元中的。一个实数表示为指数可以有不止一种形式，如 3.14159 可以表示为 3.14159×10^0 ， 0.314159×10^1 ， 0.0314159×10^2 ， 31.4159×10^{-1} ， 314.159×10^{-2} 等，它们代表同一个值。可以看到：小数点的位置是可以在 314159 几个数字之间、之前或之后(加0)浮动的，只要在小数点位置浮动的同时改变指数的值，就可以保证它的值不会改变。由于小数点位置可以浮动，所以实数的指数形式称为**浮点数**。

浮点数类型包括**float** (单精度浮点型)、**double** (双精度浮点型)、**long double**(长双精度浮点型)。

(1) **float 型**(单精度浮点型)。编译系统为每一个**float** 型变量分配4个字节，数值以规范化的二进制数指数形式存放在存储单元中。在存储时，系统将实型数据分成小数部分和指数部分两个部分，分别存放。小数部分的小数点前面的数为0。如 3.14159 在内存中的存放形式可以用图3.11表示。

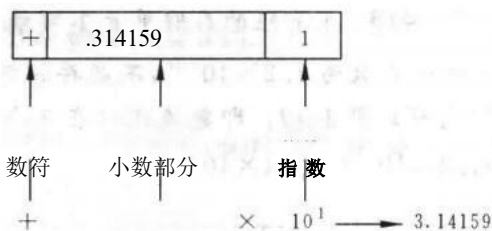


图3.11是用十进制数来示意的，实际上在计算机中是用二进制数来表示小数部分以及用2的幂次来表示指数部分的。在4个字节(32位)中，究竟用多少位来表示小数部分，多少位来表示指数部分，C标准并未具体规定，由各

C语言编译系统自定。有的C语言编译系统以24位表示小数部分(包括符号)，以8位表示指数部分(包括指数的符号)。由于用二进制形式表示一个实数以及存储单元的长度是有限的，因此不可能得到完全精确的值，只能存储成有限的精确度。小数部分占的位(bit)数愈多，数的有效数字愈多，精度也就愈高。指数部分占的位数愈多，则能表示的数值范围愈大。**float** 型数据能得到6位有效数字，数值范围为 $-3.4 \times 10^{+38} \sim 3.4 \times 10^3 8$ 。

(2) **double 型**(双精度浮点型)。为了扩大能表示的数值范围，用8个字节存储一个

① 前面已介绍：127个基本字符用7个二进制位存储，如果系统只提供127个字符，那么就将**char** 型变量的第1个二进制位设置为0，用后面7位存放127个字符的代码。在这种情况下，系统提供的**char** 类型相当于**signed char**。但是在实际应用中，往往觉得127个字符不够用，希望能多提供一些可用的字符。根据此需要，有的系统提供了扩展的字符集。把可用的字符由127个扩展为255个，即扩大了一倍。怎么解决这个问题呢？就是把本来不用的第一位用起来。把**char** 变量改为**unsigned char**，即第一位并不固定设为0，而是把8位都用来存放字符代码。这样，可以存放 $2^8 - 1$ 即255个字符代码。附录A中ASCII代码的128~255部分就是某系统扩展的ASCII字符，它并不适用于所有的系统。

读者可以用以下语句检查ASCII代码从128到255部分的扩展字符。

```
unsigned char c=128;           //定义 c 为无符号字符变量
printf("%d:%c\n",c,c);        //输出 ASCII 代码为128的字符
```

观察是否输出附录A 中代码为128的字符。可以用类似方法检查其他扩展字符。

在中文操作系统下，ASCII 代码为127以后的部分被作为中文字符处理，故不会显示出附录 A 中的扩展字符。

double 型数据，可以得到15位有效数字，数值范围为 $-1.7\times10^{-308}\sim1.7\times10^{308}$ 。为了提高运算精度，在C 语言中进行浮点数的算术运算时，将 float 型数据都自动转换为 double 型，然后进行运算。

(3) long double型(长双精度)型，不同的编译系统对long double型的处理方法不同，Turbo C对 long double 型分配16个字节。而Visual C++ 则对 long double 型和double 型一样处理，分配8个字节。请读者在使用不同的编译系统时注意其差别。

表3.4列出实型数据的有关情况 (Visual C++环境下)。

表3.4 实型数据的有关情况

类 型	字节数	有效数字	数值范围(绝对值)
float	4	6	0以及 $1.2\times10^{-38}\sim3.4\times10^38$
double	8	15	0以及 $2.3\times10^{-308}\sim1.7\times10^{308}$
long double	8	15	0以及 $2.3\times10^{-308}\sim1.7\times10^{308}$
	16	19	0以及 $3.4\times10^{-4932}\sim1.1\times10^{4932}$

说明：用有限的存储单元不可能完全精确地存储一个实数，例如 float 型变量能存储的最小正数为 1.2×10^{-38} ，不能存放绝对值小于此值的数，例如10-40。float 型变量能存储的范围见图3.12。即数值可以在3个范围内：(1) $-3.4\times10^38\sim-1.2\times10^{-38}$ ；(2) 0；(3) $1.2\times10^{-38}\sim3.4\times10^38$ 。

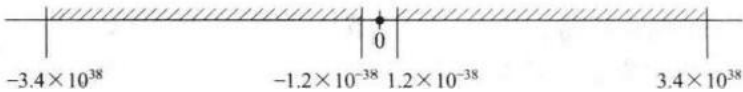


图 3.12

3.2.6 怎样确定常量的类型

在 C 语言中，不仅变量有类型，常量也有类型。为什么要把常量分为不同的类型呢？在程序中出现的常量是要存放在计算机中的存储单元中的。这就必须确定分配给它多少字节，按什么方式存储。例如，程序中有整数12, 在 Visual C++中会分配给它4个字节，按补码方式存储。

怎样确定常量的类型呢？从常量的表示形式即可以判定其类型。对于字符常量很简单，只要看到由单撇号括起来的单个字符或转义字符就可以知道它是字符常量。对于数值常量按以下规律判断。

整型常量。不带小数点的数值是整型常量，但应注意其有效范围。如在Turbo C中，系统为整型数据分配2个字节，其表值范围为-32768~32767, 如果在程序中出现数值常

量23456, 系统把它作为int 型处理, 用2个字节存放。如果出现49875, 由于超过32767, 2个字节放不下, 系统会把它作为长整型(long int)处理, 分配4个字节。在Visual C++中, 在范围-2147483648~2147483647的不带小数点的数都作为int 型, 分配4个字节, 在此范围外, 而又在 long long型数的范围内的整数, 作为long long 型处理。

在一个整数的末尾加大写字母 L 或小写字母 l, 表示它是长整型 (long int)。例如 123L, 234l 等。但在 Visual C++ 中由于对 int 和 long int 型数据都分配 4 个字节, 因此没有必要用 long int 型。

浮点型常量。凡以小数形式或指数形式出现的实数均是浮点型常量, 在内存中都以指数形式存储。如, 10 是整型常量, 10.0 是浮点型常量。那么对浮点型常量是按单精度处理还是按双精度处理呢? C 编译系统把浮点型常量都按双精度处理, 分配 8 个字节。

注意: C 程序中的实型常量都作为双精度浮点型常量。

如果有

```
float a=3.14159;
```

在进行编译时, 对 float 变量分配 4 个字节, 但对于浮点型常量 3.14159, 则按双精度处理, 分配 8 个字节。编译系统会发出“警告”(warning: truncation from 'const double' to 'float')。意为“把一个双精度常量转换为 float 型”, 提醒用户注意这种转换可能损失精度。这样的“警告”, 一般不会影响程序运行结果的正确性, 但会影响程序运行结果的精确度。

可以在常量的末尾加专用字符, 强制指定常量的类型。如在 3.14159 后面加字母 F 或 f, 就表示是 float 型常量, 分配 4 个字节。如果在实型常量后面加大写或小写的 L, 则指定此常量为 long double 型。如:

```
float a=3.14159f;           //把此3.14159按单精度浮点常量处理, 编译时不出现“警告”
long double a =1.23L        //把此1.23作为 long double 型处理
```

 **注意:** 要区分类型与变量。

有些读者容易弄不清类型和变量的关系, 往往把它们混为一谈。应当看到它们是既有联系又有区别的两个概念。每一个变量都属于一个确定的类型, 类型是变量的一个重要的属性。变量是占用存储单元的, 是具体存在的实体, 在其占用的存储单元中可以存放数据。而类型是变量的共性, 是抽象的, 不占用存储单元, 不能用来存放数据。

例如, “大学生”是一个抽象的名词, 它代表所有大学生共有的属性(在高等学校学习的、具有正式学籍的学生), 而张方章、李四元、王建则是具体存在的大学生, 他们有姓名、家庭、成绩等。可以输出张方章的成绩, 但不能输出“大学生”的成绩。同理, 可以对一个变量赋值, 但不能向一个类型赋值。如:

```
int a; a=3;           //正确。对整型变量 a 赋值
int=3;                //错误。不能对类型赋值
```

几乎每一个程序都需要进行运算，对数据进行加工处理，否则程序就没有意义了。要进行运算，就需规定可以使用的运算符。C 语言的运算符范围很宽，把除了控制语句和输入输出以外几乎所有的基本操作都作为运算符处理，例如将赋值符“=”作为赋值运算符、方括号作为下标运算符等。

3.3.1 C 运算符

C 语言提供了以下运算符：

(1) 算术运算符	(+ - * / % + + - -)
(2) 关系运算符	(><==>=<!=)
(3) 逻辑运算符	(!&. &.)
(4) 位运算符	(<<>>~ A&)
(5) 赋值运算符	(=及其扩展赋值运算符)
(6) 条件运算符	(?:)
(7) 逗号运算符	(,)
(8) 指针运算符	(* 和 &)
(9) 求字节数运算符	(sizeof)
(10) 强制类型转换运算符	((类型))
(11) 成员运算符	(. 一>)
(12) 下标运算符	([])
(13) 其他	(如函数调用运算符())

本章先介绍算术运算符和赋值运算符，其余的在以后各章中陆续介绍。

3.3.2 基本的算术运算符

最常用的算术运算符见表3.5。

表 3.5 最常用的算术运算符

运算符	含 义	举例	结 果
+	正号运算符(单目运算符)	+a	a的值
-	负号运算符(单目运算符)	-a	a的算术负值
*	乘法运算符	a*b	a和b的乘积
/	除法运算符	a/b	a除以b的商
%	求余运算符	a%b	a除以b的余数
+	加法运算符	a+b	a和b的和
-	减法运算符	a-b	a和b的差

说明:

- 由于键盘无 $\times$ 号, 运算符 $\times$ 以 $*$ 代替。
- 由于键盘无 $\div$ 号, 运算符 $\div$ 以 $/$ 代替。两个实数相除的结果是双精度实数, 两个整数相除的结果为整数, 如 $5/3$ 的结果值为1, 舍去小数部分。但是, 如果除数或被除数中有一个为负值, 则舍入的方向是不固定的。例如, $-5/3$, 有的系统中得到的结果为 -1 , 在有的系统中则得到结果为 -2 。多数C 编译系统(如 Visual C++)采取“向零取整”的方法, 即 $5/3=1$, $-5/3=-1$, 取整后向零靠拢。

- %运算符要求参加运算的运算对象(即操作数)为整数,结果也是整数。如 $8\%3$,结果为2。
- 除%以外的运算符的操作数都可以是任何算术类型。

3.3.3 自增(++)、自减(--)运算符

自增(++)、自减(--)运算符的作用是使变量的值加1或减1,例如:

$++i, --i$ (在使用 i 之前,先使 i 的值加(减)1)

$i++, i--$ (在 使用 i 之后,使 i 的值加(减)1)

粗略地看, $++i$ 和 $i++$ 的作用相当于 $i=i+1$ 。但 $++i$ 和 $i++$ 的不同之处在于: $++i$ 是先执行 $i=i+1$, 再使用 i 的值;而 $i++$ 是先使用 i 的值,再执行 $i=i+1$ 。如果 i 的原值等于3,请分析下面的赋值语句:

① $j=++i;$ (i 的值先变成4,再赋给 j , j 的值为4)

② $j=i++;$ (先将 i 的值3赋给 j 的值为3,然后 i 变为4)

又例如:

```
i=3;
```

```
printf("%d",++i);
```

输出4。若改为

```
printf("%d\n",i++);
```

则输出3。

自增(减)运算符常用于循环语句中,使循环变量自动加1;也用于指针变量,使指针指向下一个地址。这些将在以后的章节中介绍。

有些专业人员喜欢在使用++或--运算符时采用一些技巧,但是往往会出现意想不到的副作用,例如 $i+++j$, 是理解为 $(i++)+j$ 还是 $i+(++j)$ 呢?程序应当清晰易读,不致引起歧义。建议谨慎使用++和--运算符,只用最简单的形式,即 $i++, i--$ 。而且把它们作为单独的表达式,而不要在一个复杂的表达式中使用++或--运算符。

3.3.4 算术表达式和运算符的优先级与结合性

用算术运算符和括号将运算对象(也称操作数)连接起来的、符合C语法规则的式子称为**C 算术表达式**。运算对象包括常量、变量、函数等。例如,下面是一个合法的C算术表达式:

```
a*b/c-1.5+'a'
```

C语言规定了运算符的优先级(例如先乘除后加减),还规定了运算符的**结合性**。

在表达式求值时,先按运算符的优先级别顺序执行,如表达式 $a-b*c, b$ 的左侧为减号,右侧为乘号,而乘号优先级高于减号,因此,相当于 $a-(b*c)$ 。

如果在一个运算对象两侧的运算符的优先级别相同，如 $a-b+c$ ，则按规定的“结合方向”处理。C 语言规定了各种运算符的结合方向(结合性)，算术运算符的结合方向都是“自

左至右”，即先左后右，因此 b 先与减号结合，执行 $a-b$ 的运算，然后再执行加 c 的运算。“自左至右的结合方向”又称“左结合性”，即运算对象先与左面的运算符结合。以后可以看到有些运算符的结合方向为“自右至左”，即右结合性(例如，赋值运算符，若有 $a=b=c$ ，按从右到左顺序，先把变量 c 的值赋给变量 b ，然后把变量 b 的值赋给变量 a)。关于“结合性”的概念在其他一些高级语言中是没有的，是C语言的特点之一，希望能弄清楚。附录C列出了所有运算符以及它们的优先级别和结合性。



说明：不必死记，只要知道：算术运算符是自左至右(左结合性)，赋值运算符是自右至左(右结合性)，其他复杂的遇到时查一下即可。

3.3.5 不同类型数据间的混合运算

在程序中经常会遇到不同类型的数据进行运算，如 $5*4.5$ 。如果一个运算符两侧的数据类型不同，则先自动进行类型转换，使二者成为同一种类型，然后进行运算。整型、实型、字符型数据间可以进行混合运算。规律为：

(1) $+$ 、 $-$ 、 $*$ 、 $/$ 运算的两个数中有一个数为`float`或`double`型，结果是`double`型，因为系统将所有`float`型数据都先转换为`double`型，然后进行运算。

(2) 如果`int`型与`float`或`double`型数据进行运算，先把`int`型和`float`型数据转换为`double`型，然后进行运算，结果是`double`型。

(3) 字符(`char`)型数据与整型数据进行运算，就是把字符的ASCII代码与整型数据进行运算。如： $12+'A'$ ，由于字符`A`的ASCII代码是65，相当于 $12+65$ ，等于77。如果字符型数据与实型数据进行运算，则将字符的ASCII代码转换为`double`型数据，然后进行运算。

以上的转换是编译系统自动完成的，用户不必过问。

分析下面的表达式，假设已指定 i 为整型变量，值为3， f 为`float`型变量，值为2.5， d 为`double`型变量，值为7.5。

$10+'a'+i*f-d/3$

编译时，从左至右扫描，运算次序如下：

① 进行 $10+'a'$ 的运算，`'a'`的值是整数97，运算结果为107。

② 由于 $*$ 比 $+$ 优先级高，先进行 $i*f$ 的运算。先将 i 与 f 都转成`double`型，运算结果为7.5，`double`型。

③ 整数107与 $i*f$ 的积相加。先将整数107转换成双精度数，相加结果为114.5，`double`型。

④ 进行 $d/3$ 的运算，先将3转换成`double`型， $d/3$ 结果为2.5，`double`型。

⑤ 将 $10+'a'+i*f$ 的结果114.5与 $d/3$ 的商2.5相减，结果为112.0，`double`型。

【例3.3】给定一个大写字母，要求用小写字母输出。

解题思路：前已介绍，字符数据以ASCII码存储在内存中，形式与整数的存储形式相同。所以字符型数据和其他算术型数据之间可以互相赋值和运算。

要进行大小写字母之间的转换，就要找到一个字母的大写形式和小写形式之间有什么内在联系。从附录 A 中可以找到其内在规律：同一个字母，用小写表示的字符的ASCII 代码比用大写表示的字符的 ASCII 代码大32。例如字符' a' 的 ASCII 代码为97, 而' A' 的

ASCII 代码为65。将'A'的 ASCII 代码加32,就能得到'a'的 ASCII 代码。有此思路就可以编写程序了。

编写程序：

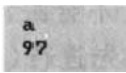
```
#include<stdio.h>

int  main()

    char  c1,c2;
    c1='A';

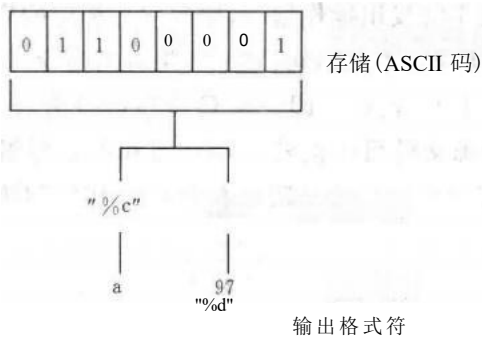
    c2=c1+32;                                     //将字符'A'的 ASCII 代码放到c1 变量中
    printf("%c\n",c2);                             //得到字符'a'的 ASCII 代码，放在c2 变量中
    printf("%d\n",c2);                             //输出c2 的值，是一个字符
    return  0;                                     //输出c2 的值，是字符'a'的 ASCII 代码
```

运行结果：



S 程序分析：程序第6行“c2=c1+32;” 把字符变量c1 的值(是字符'A'的 ASCII 代码)与整数32相加。c1+32 就是'A'+32, 就是65+32,其值为97。将97赋给字符变量c2,在 c2 的存储单元中存放了97(以二进制形式存储)。

一个字符数据既可以以字符形式输出，也可以以整数形式输出。第7行的目的是以字符形式输出c2,在 printf 函数中指定用“%c”格式，系统会将c2 变量的值97转换成相应字符'a',然后输出。最后一行的目的是以ASCII 码(十进制整数)形式输出c2 的值，故指定用“% d”输出格式，得到97,见图3. 13。



显示结果

图 3. 13

3.3.6 强制类型转换运算符

可以利用强制类型转换运算符将一个表达式转换成所需类型。例如：

```
(double)a      ( 将a 转换成 double 型 )
(int)(x+y)     ( 将 x+y  的值转换成 int 型 )
(float)(5%3)   ( 将 5 % 3的值转换成float 型 )
```

其 一般形式为

(类型名)(表达式)

注意, 表达式应该用括号括起来。如果写成

`(int)x+y`

则只将 `x` 转换成整型, 然后与 `y` 相加。

需要说明的是, 在强制类型转换时, 得到一个所需类型的中间数据, 而原来变量的类型未发生变化。例如:

`a=(int)x`

如果已定义 `x` 为 `float` 型变量, `a` 为整型变量, 进行强制类型运算 `(int)x` 后得到一个 `int` 类型的临时值, 它的值等于 `x` 的整数部分, 把它赋给 `a`, 注意 `x` 的值和类型都未变化, 仍为 `float` 型。该临时值在赋值后就不再存在了。

从上可知, 有两种类型转换。一种是在运算时不必用户干预, 系统自动进行的类型转换, 如 `3+6.5`。另一种是强制类型转换。当自动类型转换不能实现目的时, 可以用强制类型转换。如 `%` 运算符要求其两侧均为整型量, 若 `x` 为 `float` 型, 则 `x%3` 不合法, 必须用 `(int)x%3`。从附录 C 可以查到, 强制类型转换运算优先于 `%` 运算, 因此先进行 `(int)x` 的运算, 得到一个整型的中间变量, 然后再对 3 求余。此外, 在函数调用时, 有时为了使实参与形参类型一致, 可以用强制类型转换运算符得到一个所需类型的参数。

3.4 C 语 句

3.4.1 C 语句的作用和分类

在前面的例子中可以看到: 一个函数包含**声明部分**和**执行部分**, 执行部分是由语句组成的, 语句的作用是向计算机系统发出操作指令, 要求执行相应的操作。一个 C 语句经过编译后产生若干条机器指令。声明部分不是语句, 它不产生机器指令, 只是对有关数据的声明。

C 程序结构可以用图 3.14 表示。即一个 C 程序可以由若干个源程序文件(编译时以文件模块为单位)组成, 一个源文件可以由若干个函数和预处理指令以及全局变量声明部分组成(关于“全局变量”见第 7 章)。一个函数由数据声明部分和执行语句组成。

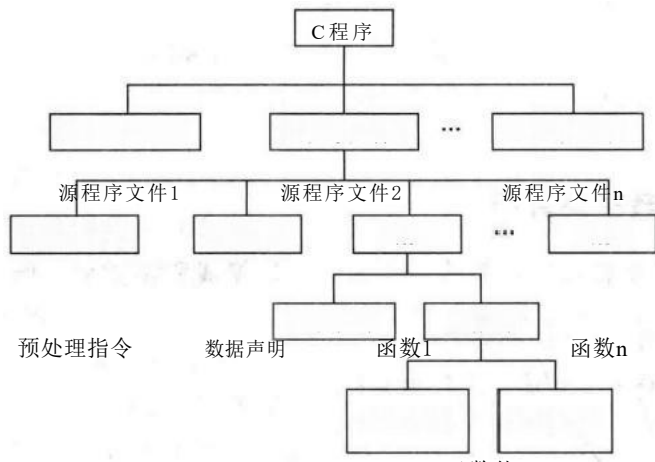


图 3.14

C 语句分为以下5类。

(1) **控制语句**。控制语句用于完成一定的控制功能。C 语言只有9种控制语句，它们的形式是：

- ① **if()...else ...** (条件语句)
- ② **for() ...** (循环语句)
- ③ **while() ...** (循环语句)
- ④ **do...while()** (循环语句)
- ⑤ **continue** (结束本次循环语句)
- ⑥ **break** (中止执行switch 或循环语句)
- ⑦ **switch** (多分支选择语句)
- ⑧ **return** (从函数返回语句)
- ⑨ **goto** (转向语句，在结构化程序中基本不用goto 语句)

上面9种语句表示形式中的()表示括号中是一个“判别条件”，“...”表示内嵌的语句。例如上面的“if()...else...”的具体语句可以写成

```
if(x>y)    z=x;else    z=y;
```

其中， $x>y$ 是一个“判别条件”，“ $z=x;$ ”和“ $z=y;$ ”是C 语句，这两个语句是内嵌在if...else 语句中的。这个if...else 语句的作用是：先判别条件“ $x>y$ ”是否成立，如果 $x>y$ 成立，就执行内嵌语句“ $z=x;$ ”，否则就执行内嵌语句“ $z=y;$ ”。

(2) **函数调用语句**。函数调用语句由一个函数调用加一个分号构成，例如：

```
printf("This is a C statement.");
```

其中printf("This is a C statement.")是一个函数调用，加一个分号成为一个语句。

(3) **表达式语句**。表达式语句由一个表达式加一个分号构成，最典型的是由赋值表达式构成一个赋值语句。例如：

```
a=3
```

是一个赋值表达式，而

```
a=3;
```

是一个赋值语句。可以看到，一个表达式的最后加一个分号就成了一个语句。一个语句必须在最后有一个分号，分号是语句中不可缺少的组成部分，而不是两个语句间的分隔符号。例如：

```
i=i+1    (是表达式，不是语句)
```

```
i=i+1;    (是语句)
```

任何表达式都可以加上分号而成为语句，例如：


```
i++;
```

是一个语句，作用是使i 值加1。又例如：

```
x+y;
```

也是一个语句,作用是完成 $x+y$ 的操作,它是合法的,但是并不把 $x+y$ 的和赋给另一变量,所以它并无实际意义。

表达式能构成语句是C 语言的一个重要特色。其实“函数调用语句”也属于表达式语句,因为函数调用(如 $\sin(x)$) 也属于表达式的一种。只是为了便于理解和使用,才把“函数调用语句”和“表达式语句”分开来说明。

(4) 空语句。 下面是一个空语句:

:

此语句只有一个分号,它什么也不做。那么它有什么用呢?可以用来作为流程的转向点(流程从程序其他地方转到此语句处),也可用来作为循环语句中的循环体(循环体是空语句,表示循环体什么也不做)。

(5) 复合语句。 可以用{} 把一些语句和声明括起来成为复合语句(又称语句块)。例如下面是一个复合语句:

```
{  
    float          pi=3.14159,r=2.5,area;           //定义变量  
    area=pi*r*r;  
    printf("area=%f",area);
```

可以在复合语句中包含声明部分(如上面的第2行),C99 允许将声明部分放在复合语句中的任何位置,但习惯上把它放在语句块开头位置。复合语句常用在 if 语句或循环中,此时程序需要连续执行一组语句。

注意: 复合语句中最后一个语句末尾的分号不能忽略不写。

3.4.2 最基本的语句——赋值语句

在 C 程序中,最常用的语句是:赋值语句和输入输出语句。其中最基本的是赋值语句。程序中的计算功能大部分是由赋值语句实现的,几乎每一个有实用价值的程序都包括赋值语句。有的程序中的大部分语句都是赋值语句。本节先介绍赋值语句,下一节介绍程序的输入输出。

先分析一个例子。

【例3.4】 给出三角形的三边长,求三角形面积。

解题思路: 假设给定的三个边符合构成三角形的条件:任意两边之和大于第三边。解此题的关键是要找到求三角形面积的公式。从数学知识已知求三角形面积的公式为

$$\text{area} = \sqrt{s(s-a)(s-b)(s-c)}$$

其中, $s=(a+b+c)/2$ 。

编写程序: 根据上面的公式编写程序如下:

```
#include<stdio.h>
```

```
#include<math.h>
```

```
int main()
```

```
{
```

```

double    a,b,c,s,area;
a=3.67;
b=5.43;
c=6.21;
s=(a+b+c)/2;
area=sqrt(s*(s-a)*(s-b)*(s-c));

printf("a=%f\tb=%f\t%f\n",a,b,c);
printf("area=%f\n",area);

return 0;

```

//定义各变量，均为double 型
//对边长a 赋值
//对边长b 赋值
//对边长c 赋值
//计算 s
//计算 area
//输出三边a,b,c 的 值
//输出面积area 的 值

运行结果：

```

a=3.670000
area=9.903431      b=5.430000      C=6.210000

```

S 程序分析：程序执行部分主要由赋值语句构成，分别实现对a,b,c 的赋值，计算 s 和 area。为了提高精度，几个变量全部定义为双精度型。第10行中 sqrt 函数是求平方根的函数。由于要调用数学函数库中的函数，必须在程序的开头加一条#include 指令，把头文件“math.h”包含到程序中来。

printf 函数双撇号内字符串中的'\t' 是转义字符，在表3.1中可以查到，它的作用是“使输出位置跳到下一个Tab 位置”。分析 printf 函数的输出情况：先原样输出字符a=，然后按 %f 格式输出变量a 的值，这时输出了“a=3.670000”，共10个字符，然后遇到\t，输出位置就跳到下一个Tab 区。一个Tab 区有8列，在输出“a=3.670000” 后已进入第2个 Tab 区，今要求跳到下一个Tab 区，就应该跳到第3个 Tab 区，即从17列开始的区。然后接着输出其后的数据。所以从第17列开始输出“b=5.430000”，再遇到\t，使输出位置又移到第5个Tab 区，从第33列开始输出“c=6.210000”。

在安排输出时，常用\t 来调整输出的位置，使输出的数据清晰、整齐、美观。

注意：以后凡在程序中要用到数学函数库中的函数，都应当在本文件的开头包含 math.h 头文件。

下面归纳一下与赋值有关的一些问题。

1. 赋值运算符

赋值符号“=”就是赋值运算符，它的作用是将一个数据赋给一个变量。如a=3 的作用是执行一次赋值操作(或称赋值运算)。把常量3赋给变量a。也可以将一个表达式的值赋给一个变量。

*2. 复合的赋值运算符

在赋值符=之前加上其他运算符，可以构成复合的运算符。如果在“=”前加一个“+”运算符就成了复合运算符“+=”。例如，可以有以下的复合赋值运算：

$a+=3$	等价于	$a=a+3$
$X*=y+8$	等价于	$x=x*(y+8)$
$x\%=3$	等价于	$x=x\%3$

以“ $a+=3$ ”为例来说明,它相当于使 a 进行一次自加3的操作。即:先使 a 加3,再赋给 a 。同样,“ $x*=y+8$ ”的作用是使 x 乘以 $(y+8)$,再赋给 x 。

为便于理解和记忆,可以这样理解 $a+=b$:

- ① $a+=b$ (其中 a 为变量, b 为表达式)
- ② $\underline{a+}=b$ (将有下画线的“ $a+$ ”移到右侧)
- ③ $a=\underline{a+b}$ (在右侧补上变量名 a)

注意: 如果 b 是包含若干项的表达式,则相当于它有括号。例如,以下3种写法是等价的:

- ① $x\%=y+3$
- ② $x=(y+3)$
- ③ $x=x\%(y+3)$ (不要错写成 $x=x\%y+3$)

凡是二元(二目)运算符,都可以与赋值符一起组合成复合赋值符。有关算术运算的复合赋值运算符有 $+=$, $=$, $*=$, $/=$, $\%=$ 。

C 语言采用这种复合运算符,一是为了简化程序,使程序精练,二是为了提高编译效率,能产生质量较高的目标代码。专业人员往往喜欢使用复合运算符,程序显得专业一点。对初学者来说,不必多用,首要的是保持程序清晰易懂。本节在此作简单的介绍,是为了便于读者阅读别人编写的程序。对本小节内容有一定了解即可。

3. 赋值表达式

前面介绍过,赋值语句是在赋值表达式的末尾加一个分号构成的。那么什么是赋值表达式呢?

由赋值运算符将一个变量和一个表达式连接起来的式子称为“赋值表达式”。它的一般形式为

变量 赋值运算符 表达式

赋值表达式的作用是将一个表达式的值赋给一个变量,因此赋值表达式具有计算和赋值的双重功能。如 $a=3*5$ 是一个赋值表达式。对赋值表达式求解的过程是:先求赋值运算符右侧的“表达式”的值,然后赋给赋值运算符左侧的变量。既然是一个表达式,就应该有一个值,表达式的值等于赋值后左侧变量的值。例如,赋值表达式 $a=3*5$,对表达式求解后,变量 a 的值和表达式的值都是15。

赋值运算符左侧应该是一个可修改值的“左值”(left value,简称为lvalue)。左值的意思是它可以出现在赋值运算符的左侧,它的值是可以改变的。并不是任何形式的数据都可以作为左值的,左值应当为存储空间并可以被赋值。变量可以作为左值,而算术表达式 $a+$

b 就不能作为左值，常量也不能作为左值，因为常量不能被赋值。能出现在赋值运算符右侧的表达式称为“右值”(right value,简写为rvalue)。显然左值也可以出现在赋值运算符右侧，因而凡是左值都可以作为右值。例如：

b=a;

//b 是左值

```
c=b;           //b 也是右值
```

赋值表达式中的“表达式”又可以是一个赋值表达式。例如：

```
a=(b=5)
```

括号内的**b=5** 是一个赋值表达式，它的值等于5。执行表达式“**a=(b=5)**”，就是执行**b=5** 和 **a=b** 两个赋值表达式。因此**a** 的值等于5，整个赋值表达式的值也等于5。从附录 C 可知赋值运算符按照“自右而左”的结合顺序，因此，(b=5) 外面的括号可以不要，即**a=(b=5)** 和 **a=b=5** 等价，都是先求 **b=5** 的值(得5)，然后再赋给**a**，下面是赋值表达式的例子：

```
a=b=c=5           (赋值表达式的值为5, a,b,c 值均为5)
a=5+(c=6)         (表达式值为11, a 值为11, c 值为6)
a=(b=4)+(c=6)      (表达式值为10, a 值为10, b 等于4, c 等于6)
a=(b=10)/(c=2)     (表达式值为5, a 等于5, b 等于10, c 等于2)
```

请分析下面的赋值表达式：

```
a=(b=3*4)
```

将**3*4**的值先赋给变量**b**，然后把变量**b**的值赋给变量**a**，最后**a**和**b**的值都等于12。

把赋值表达式作为表达式的一种，使得赋值操作不仅可以出现在赋值语句中，而且可以以表达式的形式出现在其他语句中(如输出语句、循环语句等)，如：

```
printf("%d",a=b);
```

如果**b**的值为3，则输出**a**的值(也是表达式**a=b**的值)为3。在一个**printf**函数中完成了赋值和输出双重功能。这是C语言灵活性的一种表现。以后将进一步看到这种应用及其优越性。

*4. 赋值过程中的类型转换

如果赋值运算符两侧的类型一致，则直接进行赋值。如：

```
i=234;           //设已定义i 为整型变量
```

此时直接将整数234存入变量**i**的存储单元中。

如果赋值运算符两侧的类型不一致，但都是基本类型时，在赋值时要进行类型转换。类型转换是由系统自动进行的，转换的规则是：

(1) 将浮点型数据(包括单、双精度)赋给整型变量时，先对浮点数取整，即舍弃小数部分，然后赋予整型变量。如果**i**为整型变量，执行“**i=3.56;**”的结果是使**i**的值为3，以整数形式存储在整型变量中。

(2) 将整型数据赋给单、双精度变量时，数值不变，但以浮点数形式存储到变量中。如果有**float**变量**f**，执行“**f=23;**”。先将整数23转换成实数23.0，再按指数形式存储在变量**f**中。如将23赋给**double**型变量**d**，即执行“**d=23;**”，则将整数23转换成双精度实数23.0，然后以双精度浮点数形式存储到变量**d**中。

(3) 将一个double 型数据赋给float 变量时，先将双精度数转换为单精度，即只取6~7位有效数字，存储到 float 型变量的4个字节中。应注意双精度数值的大小不能超出float 型

变量的数值范围。例如, 将一个 `double` 型变量 `d` 中的双精度实数赋给一个 `float` 型变量 `f`。

```
double d=123.456789e100;           //指数为100,超过了float数据的最大范围
f=d;
```

`f` 无法容纳如此大的数, 就出现错误, 无法输出正确的信息。

将一个 `float` 型数据赋给 `double` 型变量时, 数值不变, 在内存中以8个字节存储, 有效位数扩展到15位。

(4) 字符型数据赋给整型变量时, 将字符的 ASCII 代码赋给整型变量。如:

```
i='A';                               //已定义i为整型变量
```

由于 'A' 字符的 ASCII 代码为65, 因此赋值后 `i` 的值为65。

(5) 将一个占字节多的整型数据赋给一个占字节少的整型变量或字符变量(例如把占4个字节的 `int` 型数据赋给占2个字节的 `short` 变量或占1个字节的 `char` 变量)时, 只将其低字节原封不动地送到被赋值的变量(即发生“截断”)。例如:

```
int i=289;
char c='a';
c=i;
```

赋值情况见图3.15。 `c` 的值为33, 如果用 “`%c`” 输出 `c`, 将得到字符 “!” (其 ASCII 码为33)。

又如:

```
int a=32767;
short b;
b=a+1;
```

按理论上应得到32768, 但输出的结果却是一32768, 看上去莫名其妙, 其实原因很简单, 对短整型数据分配2个字节, 最大能表示32767, 无法表示32768, 见图3.16。

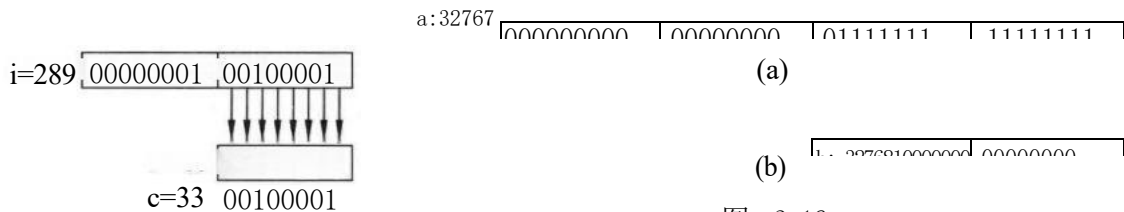


图 3.16

图3.16(a) 表示 `int` 型变量用4个字节存储32767的情况, 加1以后, 两个低字节的第1位为1, 后15位为0, 把它传送到 `short` 变量 `b` 中, 见图3.16(b)。由于整型变量的最高位代表符号, 第1位是1, 代表此数是负数, 它就是一32768的补码形式。对一般初学者来说, 只需要知道变量的数值范围即可。

要避免把占字节多的整型数据向占字节少的整型变量赋值, 因为赋值后数值可能发生失真。如果一定要进行这种赋值, 应当保证赋值后数值不会发生变化, 即所赋的值在变量的允许数值范围内。如果把上面的 `a` 值改为12345, 就不会失真。

以上的赋值规则看起来比较复杂，其实，不必死记。只要知道整型数据之间的赋值，按存储单元中的存储形式直接传送。实型数据之间以及整型与实型之间的赋值，是先转换(类

型)后赋值。

由于C语言使用灵活,在不同类型数据之间赋值时,常常会出现数据的失真,而且这不属于语法错误,编译系统并不提示出错,全靠程序员的经验来找出问题。这就要求编程人员对出现问题的原因有所了解,以便迅速排除故障。

5. 赋值表达式和赋值语句

在C程序中,赋值语句是用得最多的语句。在3.4.1节的C语句分类中,并没有看到赋值语句,实际上,C语言的赋值语句属于表达式语句,由一个赋值表达式加一个分号组成。其他一些高级语言(如BASIC,FORTRAN, COBOL,Pascal等)有赋值语句,而无“赋值表达式”这一概念。这是C语言的一个特点,使之应用灵活方便。

前面已经提到,在一个表达式中可以包含另一个表达式。赋值表达式既然是表达式,那么它就可以出现在其他表达式之中。例如:

```
if((a=b)>0)      max=a;
```

按一般理解,if后面的括号内应该是一个“条件”,例如可以是

```
if(a>0)      max=a;
```

现在,在a的位置上换上一个赋值表达式a=b,其作用是:先进行赋值运算(将b的值赋给a),然后判断a是否大于0,如大于0,执行max=a。请注意,在if语句中的a=b不是赋值语句,而是赋值表达式。如果写成

```
if((a=b;)>0)      max=a;           // “a=b;” 是赋值语句
```

就错了。在if的条件中可以包含赋值表达式,但不能包含赋值语句。由此可以看到,C语言把赋值语句和赋值表达式区别开来,增加了表达式的种类,使表达式的应用几乎“无孔不入”,能实现其他语言中难以实现的功能。

注意:要区分赋值表达式和赋值语句。

赋值表达式的末尾没有分号,而赋值语句的末尾必须有分号。在一个表达式中可以包含一个或多个赋值表达式,但绝不能包含赋值语句。

6. 变量赋初值

从前面的程序中可以看到:可以用赋值语句对变量赋值,也可以在定义变量时对变量赋以初值。这样可以使程序简练。如:

```
int    a=3;
float  f=3.56;           //指定 a 为整型变量,初值为3
char   c='a';           //指定 f 为浮点型变量,初值为3.56
//指定 c 为字符变量,初值为'a'
```

也可以使被定义的变量的一部分赋初值。例如:

```
int    a,b,c=5;
```

指定 `a,b,c` 为整型变量，但只对 `c` 初始化，`c` 的初值为5。
如果对几个变量赋予同一个初值，应写成

```
int      a=3,b=3,c=3;
```

表示 a, b, c 的初值都是3。不能写成

```
int  a=b=c=3;
```

一般变量初始化不是在编译阶段完成的(只有在静态存储变量和外部变量的初始化是在编译阶段完成的),而是在程序运行时执行本函数时赋予初值的,相当于执行一个赋值语句。例如:

```
int  a=3;
```

相当于

```
int  a;
a=3;
```

//指定 a 为整型变量

//赋值语句,将3赋给 a

又如:

```
int  a,b,c=5;
```

相当于

```
int  a,b,c;
c=5;
```

//指定 a, b, c 为整型变量

//将5赋给 c

3.5 数据的输入输出

3.5.1 输入输出举例

前面已经看到了利用 `printf` 函数进行数据输出的程序,现在再介绍一个包含输入和输出的程序。

【例3.5】求 $ax^2+bx+c=0$ 方程的根。 a, b, c 由键盘输入, 设 $b^2-4ac>0$ 。

解题思路: 首先要知道求方程式的根的方法。由数学知识已知: 如果 $b^2-4ac\geq 0$, 则一元二次方程有两个实根:

$$x_1 = \frac{-b + \sqrt{b^2 - 4ac}}{2a}, \quad x_2 = \frac{-b - \sqrt{b^2 - 4ac}}{2a}$$

可以将上面的分式分为两项:

$$p = \frac{-b}{2a}, \quad q = \frac{\sqrt{b^2 - 4ac}}{2a}$$

则

$$T_1=p+q, \quad x_2=p-q$$

有了这些式子，只要知道a,b,c 的值，就能顺利地求出方程的两个根。

剩下的问题就是输入a,b,c 的值和输出根的值了。需要用scanf 函数输入a,b,c 的值，用 printf 函数输出两个实根的值。

编写程序：

```
#include<stdio.h>

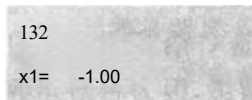
#include <math.h>
int main()
{double      a,b,c,disc,x1,x2,p,q;
scanf("%1f%1f%1f",&a,&b,&c);
disc=b*b-4*a*c;

p=-b/(2.0*a);
q=sqrt(disc)/(2.0*a);
x1=p+q;x2=p-q;

printf("x1=%7.2f\nx2=%7.2f",x1,x2);
return 0;

//程序中要调用求平方根函数 sqrt
//disc 用来存放判别式(b*b—4ac) 的值
//输入双精度型变量的值要用格式声明" %1f"
//求出方程的两个根
//输出方程的两个根
```

运行结果：



```
132
x1= -1.00
```

注意在输入数据时，1, 3, 2这3个数之间用空格分隔，最后按“回车”键。

程序分析：

(1)用scanf 函数输入a,b,c 的值，请注意在scanf 函数中括号内变量a,b,c 的前面，要用地址符&，即&. a,&b,&c。&a 表示变量a 在内存中的地址。该 scanf 函数表示从终端输入的3个数据分别送到地址为&. a,&b,&c 的存储单元，也就是赋给变量a,b,c。双撇号内用%1f格式声明，表示输入的是双精度型实数。

(2)在scanf 函数中，格式声明为“%1f%1f%1f”，连续3个“%1f”。要求输入3个双精度实数。请注意在程序运行时应怎样输入数据。从上面运行情况中可以看到输入“13 2”，两个数之间用空格分开。这是正确的，如果用其他符号(如逗号)会出错。现在输入的是整数，但由于指定用%1f格式输入，因此系统会先把这3个整数转换成实数1.0, 3.0, 2.0, 然后赋给变量a,b,c。

(3)在printf 函数中，不是简单地用%f格式声明，而是在格式符f的前面加了“7.2”，表示在输出x1 和 x2 时，指定数据占7列，其中小数占2列。请分析运行结果。这样做的好处是：①可以根据实际需要来输出小数的位数，因为并不是任何时候都需要6位小数的，例如价格只须2位小数即可(第3位按四舍五入处理)。②如果输出多个数据，各占一行，而用同一个格式声明(如%7.2f)，即使输出的数据整数部分值不同，但输出时上下行必然按小数点对齐，使输出数据整齐美观。读者可自己试一下。

(4)在本例中假设给定的a,b,c 的值满足 $b^2-4ac>0$ ，所以程序不对此进行判断。在实际上，用所输入的a,b,c 并不一定能求出两个实根。因此为稳妥起见，应在程序的开头检

查 b^2-4ac 是否大于等于0。只有确认它大于等于0, 才能用上述方法求方程的根。在学习了下一章后, 就可以用if 语句来进行检查了。

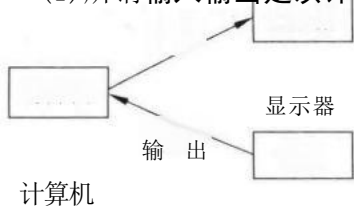
3.5.2 有关数据输入输出的概念

从前面的程序可以看到：几乎每一个C 程序都包含输入输出。因为要进行运算，就必

须给出数据,而运算的结果当然需要输出,以便人们应用。没有输出的程序是没有意义的。输入输出是程序中最基本的操作之一。

在讨论程序的输入输出时首先要注意以下几点。

(1)所谓输入输出是以计算机主机为主体而言的。从计算机向输出设备(如显示器、打



印机等)输出数据称为输出,从输入设备(如键盘、光盘、扫描仪等)向计算机输入数据称为输入,如图3.17所示。

图 3.17

(2) C 语言本身不提供输入输出语句,输入和输出操作是由 C 标准函数库中的函数来实现的。在 C 标准函数库中提供了一些输入输出函数,例如 `printf` 函数和 `scanf` 函数。读者在使用它们时,千万不要误认为它们是 C 语言提供的“输入输出语句”。`printf` 和 `scanf` 不是 C 语言的关键字,而

只是库函数的名字。实际上可以不用 `printf` 和 `scanf` 这两个名字,而另外编写一个输入函数和一个输出函数,用来实现输入输出的功能,采用其他名字作为函数名。

C 提供的标准函数以库的形式在 C 的编译系统中提供,它们不是 C 语言文本中的组成部分。不把输入输出作为 C 语句的目的是使 C 语言编译系统简单精练,因为将语句翻译成二进制的指令是在编译阶段完成的,没有输入输出语句就可以避免在编译阶段处理与硬件有关的问题,可以使编译系统简化,而且通用性强,可移植性好,在各种型号的计算机和不同的编译环境下都能适用,便于在各种计算机上实现。

各种 C 编译系统提供的系统函数库是各软件公司编制的,它包括了 C 语言建议的全部标准函数,还根据用户的需要补充一些常用的函数,已对它们进行了编译,成为目标文件(.obj 文件)。它们在程序连接阶段与由源程序经编译而得到的目标文件(.obj 文件)相连接,生成一个可执行的目标程序(.exe 文件)。如果在源程序中有 `printf` 函数,在编译时并不把它翻译成目标指令,而是在连接阶段与系统函数库相连接后,在执行阶段中调用函数库中的 `printf` 函数。

不同的编译系统所提供的函数库中,函数的数量、名字和功能是不完全相同的。不过,有些通用的函数(如 `printf` 和 `scanf` 等),各种编译系统都提供,成为各种系统的标准函数。

C 语言函数库中有一批标准输入输出函数,它是以标准的输入输出设备(一般为终端设备)为输入输出对象的。其中有 `putchar` (输出字符)、`getchar` (输入字符)、`printf` (格式输出)、`scanf` (格式输入)、`puts` (输出字符串)和 `gets` (输入字符串)。本章主要介绍前面4个最基本的输入输出函数。

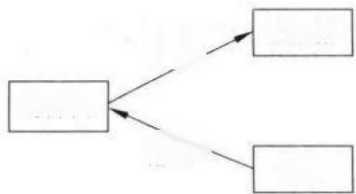
(3)要在程序文件的开头用预处理指令 `#include` 把有关头文件放在本程序中。

如:

```
#include<stdio.h>
```

如果程序调用标准输入输出函数,就必须在本程序的开头用 `#include` 指令把 `stdio.h`

头文件包含到程序中。 # include 指令放在程序的开头，所以把 stdio.h 称为“头文件”(header file),文件后缀为“.h”。在 stdio.h 头文件中存放了调用标准输入输出函数时所需要的信息，包括与标准I/O 库有关的变量定义和宏定义以及对函数的声明。在对程序进行



编译预处理时，系统会把在该头文件中存放的内容调出来，取代本行的#include 指令。这些内容就成为了程序中的一部分。调用不同的库函数，应当把不同的头文件包含进来，见本书附录 E(C 库函数)。

说明： #include 指令还有一种形式，头文件不是用尖括号括起来，而是用双撇号，如：

```
#include"stdio.h"
```

这两种#include 指令形式的区别是：用尖括号形式(如<stdio.h>) 时，编译系统从存放 C 编译系统的子目录中去找所要包含的文件(如stdio.h)，这称为**标准方式**。如果用双撇号形式(如"stdio.h")，在编译时，编译系统先在用户的当前目录(一般是用户存放源程序文件的子目录)中寻找要包含的文件，若找不到，再按标准方式查找。如果用#include 指令是为了使用系统库函数，因而要包含系统提供的相应头文件，这时以用标准方式为宜，以提高效率。如果用户想包含的头文件不是系统提供的相应头文件，而是用户自己编写的文件(这种文件一般都存放在用户当前目录中)，这时应当用双撇号形式，否则会找不到所需的文件。如果该头文件不在当前目录中，可以在双撇号中写出文件路径(如#include "C:\temp\filel.h")，以便系统能从中找到所需的文件。

 **注意：** 应养成习惯，只要在本程序文件中使用标准输入输出库函数时，一律加上 #include <stdio.h> 指令。

3.5.3 用 printf 函数输出数据

在 C 程序中用来实现输出和输入的主要是 printf 函数和 scanf 函数。这两个函数是格式输入输出函数。用这两个函数时，程序设计人员必须指定输入输出数据的格式，即根据数据的不同类型指定不同的格式。

 **说明：** C 提供的输入输出格式比较多，也比较烦琐，初学时不易掌握，更不易记住。用得不对就得不到预期的结果，不少编程人员由于掌握不好这方面的知识而浪费了大量调试程序的时间。为了使读者便于掌握，本章主要介绍最常用的格式输入输出，有了这些基本知识，就可以顺利地进行一般的编程工作了。以后再结合应用进一步介绍格式输入输出的各种应用。

在初学时不必花许多精力去深究每一个细节，重点掌握最常用的一些规则即可。其他部分可在需要时随时查阅。学习这部分的内容时最好边看书边上机练习，通过编写和调试程序的实践逐步深入而自然地掌握输入输出的应用。

在前面的例题中已经多次用 printf 函数输出数据，下面再作比较系统的介绍。

printf 函数(格式输出函数)用来向终端(或系统隐含指定的输出设备)输出若干个任意类型的数据。

1. printf 函数的一般格式

printf 函数的一般格式为

printf (格式控制, 输出表列)

例如:

```
printf("%d,%c\n",i,c)
```

括号内包括两部分:

(1) “格式控制”是用双撇号括起来的一个字符串,称为格式控制字符串,简称格式字符串。它包括两个信息:

① 格式声明。格式声明由“%”和格式字符组成,如%d、%f等。它的作用是将输出的数据转换为指定的格式后输出。格式声明总是由“%”字符开始的。

② 普通字符。普通字符即需要在输出时原样输出的字符。例如上面 printf 函数中双撇号内的逗号、空格和换行符,也可以包括其他字符。

(2) 输出表列是程序需要输出的一些数据,可以是常量、变量或表达式。

下面是 printf 函数的具体例子:

```
printf("%d    %d\n",a,b)
printf("a=%d b=%d\n",a,b)
```

格式声明 输出表列

格式声明 输出表列

在第2个printf函数中的双撇号内的字符除了两个“%d”以外,还有非格式声明的普通字符(如 a=,b= 和 '\n'),它们全部按原样输出。如果a和b的值分别为3和4,则输出结果为

```
a=3    b=4
```

执行'\n'使输出控制移到下一行的开头,从显示屏幕上可以看到光标已移到下一行的开头。

上面输出结果中有下画线的字符是printf函数中的“格式控制字符串”中的普通字符按原样输出的结果。3和4是a和b的值(注意3和4这两个数字前和后都没有加空格),其数字位数由a和b的值而定。假如a=12,b=123,则输出结果为

```
a= 12    b= 123
```

由于printf是函数,因此,格式控制字符串和输出表列实际上都是函数的参数。

printf函数的一般形式可以表示为

```
printf (参数1, 参数2, 参数3, ..., 参数n)
```

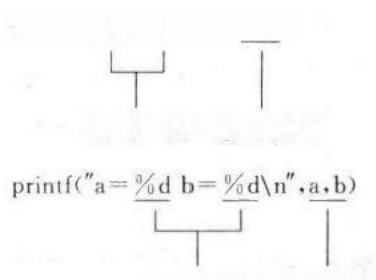
参数1是格式控制字符串,参数2~参数n是需要输出的数据。执行printf函数时,将参数2~参数n按参数1所指定的格式进行输出。参数1是必须有的,参数2~参数n是可选的。

2. 格式字符

前已介绍，在输出时，对不同类型的数据要指定不同的格式声明，而格式声明中最重要的是格式字符。常用的有以下几种格式字符。

(1) **d 格式符**。用来输出一个有符号的十进制整数。

在前面的例子中已经看到了：在输出时，按十进制整型数据的实际长度输出，正数的符号不输出。



可以在格式声明中指定输出数据的**域宽**(所占的列数),如用“%5d”,指定输出数据占5列,输出的数据显示在此5列区域的右侧。如:

```
printf("%5d\n%5d\n",12,—345);
```

输出结果为

```
12      (12前面有3个空格)
-345    (-345前面有1个空格)
```

若输出long (长整型)数据,在格式符d前加字母l(代表long),即“%ld”。若输出long long (双长整型)数据,在格式符d前加两个字母l(代表long long),即“%lld”。

(2) **c 格式符**。用来输出一个字符。例如:

```
char ch='a';
printf("%c",ch);
```

运行时输出

```
a
```

也可以指定域宽,如

```
printf("%5c",ch);
```

运行时输出

```
a
```

(a前面有4个空格)

一个整数,如果在0~127范围中,也可以用“%c”使之按字符形式输出,在输出前,系统会将该整数作为ASCII码转换成相应的字符;如:

```
short a=121;
printf("%c",a);
```

输出字符y。如果整数比较大,则把它的最后一个字节的信息以字符形式输出。如:

```
int a=377;
printf("%c",a);
```

也输出字符y,见图3.18。因为用%c格式输出时,只考虑一个字节,存放a的存储单元中最后一个字节中的信息是01111001,即十进制的

121,它是'y'的ASCII代码。

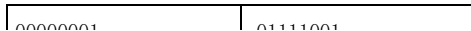


图 3.18

(3) **s 格式符**。用来输出一个字符串。如:

```
printf("%s","CHINA");
```

执行此函数时在显示屏上输出字符串“CHINA”(不包括双引号)。

(4) **f 格式符**。用来输出实数(包括单、双精度、长双精度),以小数形式输出,有几种用法:

①基本型,用%f。

不指定输出数据的长度，由系统根据数据的实际情况决定数据所占的列数。系统处理的方法一般是：实数中的整数部分全部输出，小数部分输出6位。

【例3.6】 用 %f 输出实数，只能得到6位小数。

```
#include<stdio.h>
int main()
{
    double a=1.0;
    printf("%f\n",a/3);
    return 0;
}
```

运行结果：

0.333333

虽然a是双精度型，a/3的结果也是双精度型，但是用%f格式声明只能输出6位小数。

②指定数据宽度和小数位数，用%m.nf。

例3.5已经用了“%7.2”格式指定了输出的数据占7列，其中包括2位小数。对其后一位采取四舍五入方法处理，即向上或向下取近似值。如果把小数部分指定为0，则不仅不输出小数，而且小数点也不输出。如果在例3.6的printf函数中指定“%7.0f”格式声明，由于其整数部分为0，因此输出结果为0。所以不要轻易指定小数的位数为0。

如果想在例3.6中输出双精度变量a的15位小数，可以采用例3.5所用的方法，用“%20.15 f”格式声明，即把上面程序的第4行改为

```
printf("%20.15f\n",a/3);
```

运行结果：

0.333333333333333

注意在0的前面有3个空格。

这时输出了15位小数。但是应该注意：一个双精度数只能保证15位有效数字的精确度，即使指定小数位数为50(如用%55.50f)，也不能保证输出的50位都是有效的数字。读者可以上机试一下。

 **注意：**在用%f输出时要注意数据本身能提供的有效数字，如float型数据的存储单元只能保证6位有效数字。double型数据能保证15位有效数字。不要以为计算机输出的所有数字都是绝对精确的。

【例3.7】 float型数据的有效位数。

```
#include<stdio.h>
int main()
{
    float a;
    a=10000/3.0;
    printf("%f\n",a);
    return 0;
}
```

运行结果:

3333.333333

本来计算的理论值应为3333.333333333..., 但由于float 型数据只能保证6~7位有效数字, 因此虽然程序输出了6位小数, 但从左面开始的第7位数字(即第3位小数)以后的数字并不保证是绝对正确的。

如果将a 改为double 型, 其他不变, 请考虑输出结果如何, 可上机一试。

③ 输出的数据向左对齐, 用 % -m.nf。

在 m.n 的前面加一个负号, 其作用与%m.nf 形式作用基本相同, 但当数据长度不超过m 时, 数据向左靠, 右端补空格。如:

```
printf("%-25.15f,%25.15f\n",a,a);
```

运行结果:

```
3333.33333333333500      3333.33333333333500
```

第1次输出 a 时输出结果向左端靠, 右端空5列。第2次输出a 时输出结果向右端靠, 左端空5列。

(5) e 格式符。用格式声明 %e 指定以指数形式输出实数。如果不指定输出数据所占的宽度和数字部分的小数位数, 许多C 编译系统(如Visual C++)会自动给出数字部分的小数位数为6位, 指数部分占5列(如e+002, 其中“e”占1列, 指数符号占1列, 指数占3列)。数值按标准化指数形式输出(即小数点前必须有而且只有1位非零数字)。例如:

```
printf("%e",123.456);
```

输出如下:

```
1.234560 e+002
 6列      5列
```

所输出的实数共占13列宽度(注: 不同系统的规定略有不同)。

也可以用“%m.ne”形式的格式声明, 如:

```
printf("%13.2e",123.456);
```

输出为

```
1.23e+002      (数的前面有4个空格)
```

格式符e 也可以写成大写E 形式, 此时输出的数据中的指数不是以小写字母 e 表示而以大写字母 E 表示, 如1.23456 E+002。

以上几种输出格式是常用的, 在以后各章中会结合实际问题加以具体应用, 读者可在实际应用中逐步掌握它们。

* (6) 其他格式符。

C 语言还提供以下几种输出格式符, 由于初学时用得不多, 不作详细介绍, 只供必要时查阅。

① i 格式符。作用与 d 格式符相同, 按十进制整型数据的实际长度输出。一般习惯用 %d 而少用%i。

② o 格式符。以八进制整数形式输出。将内存单元中的各位的值(0或1)按八进制形式输出，因此输出的数值不带符号，即将符号位也一起作为八进制数的一部分输出。

例如:

```
int a=-1;
printf("%d\t%o\n",a,a);
```

-1在内存单元中的存放形式(以补码形式存放在4个字节)如下:

11111111	11111111	11111111	11111111
----------	----------	----------	----------

运行时输出:

```
-1      377777777777
```

用 %d (十进制整数形式)输出 a 时,得到-1,按%o 输出时,按内存单元中实际的二进制数按3位一组构成八进制数形式,如上面的32个二进制位可以从右至左每3位为一组:

二进制数111就是八进制数7。因此上面的数用八进制数表示为377777777777。八进制整数是不会带负号的。用%o 格式声明可以得到存储单元中实际的存储情况。

③ x 格式符。以十六进制数形式输出整数。例如:

```
int a=-1;
printf("%d\t%o\t%x\n",a,a,a);
```

输出结果为

```
-1      377777777777      ffffffff
```

同样可以用 “%lx”输出长整型数,也可以指定输出字段的宽度,如 “%12x”。

如果读者对二进制数、八进制数、十六进制数、补码等不熟悉,可以忽略这部分内容,在需要时可参阅有关书籍,这些不属本书的范围。

④ u 格式符。用来输出无符号(unsigned) 型数据,以十进制整数形式输出。

⑤ g 格式符。用来输出浮点数,系统自动选f 格式或e 格式输出,选择其中长度较短的格式,不输出无意义的0。如:

```
double a=12345678954321;
printf("%f\t%e\t%g\n",a,a,a);
```

输出结果为

```
12345678954321.000000      1.234568e+813      1.23457e+013
```

可从以上看到用%f格式输出占21列，用%e格式输出占13列，故%g采用%e格式输出。
综合上面的介绍，格式声明的一般形式可以表示为

% 附加字符 格式字符

以上介绍的加在格式字符前面的字符(如l, m, n, 一等)就是附加字符，又称为修饰符，起补充声明的作用。

为便于查阅，表3.6和表3.7分别列出了printf函数中用到的格式字符和附加字符。

表3.6 printf函数中用到的格式字符

格式字符	说 明
d, i	以带符号的十进制形式输出整数(正数不输出符号)
o	以八进制无符号形式输出整数(不输出前导符0)
x, X	以十六进制无符号形式输出整数(不输出前导符0x), 用x则输出十六进制数的a~f时以小写形式输出, 用X时, 则以大写字母输出
u	以无符号十进制形式输出整数
c	以字符形式输出, 只输出一个字符
S	输出字符串
f	以小数形式输出单、双精度数, 隐含输出6位小数
e, E	以指数形式输出实数, 用e时指数以“e”表示(如1. 2e+02), 用E时指数以“E”表示(如1. 2E+02)
g, G	选用%f或%e格式中输出宽度较短的一种格式, 不输出无意义的0。用G时, 若以指数形式输出, 则指数以大写表示

在格式声明中, 在%和上述格式字符间可以插入表3. 7中列出的几种附加符号(又称修饰符)。

表3. 7 printf函数中用到的格式附加字符

字 符	说 明
l	长整型整数, 可加在格式符d、o、x、u前面
m(代表一个正整数)	数据最小宽度
n(代表一个正整数)	对实数, 表示输出n位小数; 对字符串, 表示截取的字符个数
—	输出的数字或字符在域内向左靠

说明:

- (1) printf 函数输出时, 务必注意输出对象的类型应与上述格式说明匹配, 否则将会出现错误。
- (2)除了X,E,G 外, 其他格式字符必须用小写字母, 如%d 不能写成%D。
- (3)可以在printf 函数中的格式控制字符串内包含转义字符, 如\n,\t,\b,\r,\f 和 \377等。

(4)表3.6中所列出的字母 d,o,x,u,c,s,f,e,g,X,E 和 G 等，如用在格式声明中就作为格式字符。 一个格式声明以 “%” 开头， 以上述12个格式字符之一为结束， 中间可以插入附加格式字符(也称修饰符)。例如：

```
printf("c= %c f= %fs= %s",c,f,s);
```



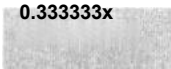
格式声明

第1个格式声明为“%c”而不包括其后的字母f；第2个格式声明为“%f”，不包括其后的字母s；第3个格式声明为“%s”。其他字符都是在输出时按原样输出的普通字符。

(5) 如果想输出字符“%”，应该在“格式控制字符串”中用连续两个“%”表示，如：

```
printf("%f%%\n",1.0/3);
```

输出：



实现了输出“%”符号。

3.5.4 用 scanf 函数输入数据

在本章例3.5程序中已经看到怎样用scanf函数输入数据。下面再作比较系统的说明。

1. scanf 函数的一般形式

```
scanf( 格式控制, 地址表列)
```

“格式控制”的含义同printf 函数。“地址表列”是由若干个地址组成的表列，可以是变量的地址，或字符串的首地址。

2. scanf 函数中的格式声明

与 printf 函数中的格式声明相似，以%开始，以一个格式字符结束，中间可以插入附加的字符。

例3.5中的scanf 函数是比较简单的。可以把scanf 函数改写成以下形式：

```
scanf("a=%f,b=%f,c=%f",&a,&b,&c);
```

在格式字符串中除了有格式声明%f以外，还有一些普通字符(有“a=”“b=”“c=” 和“,”)。

表3.8和表3.9列出scanf 函数所用的格式字符和附加字符。它们的用法和 printf 函数中的用法差不多。

*表3.8 scanf函数中用到的格式字符

格式字符	说 明
d, i	输入有符号的十进制整数
u	输入无符号的十进制整数
o	输入无符号的八进制整数
x, X	输入无符号的十六进制整数(大小写作用相同)
c	输入单个字符
S	输入字符串，将字符串送到一个字符数组中，在输入时以非空白字符开始，以第

	一个空白字符结束。字符串以串结束标志'\0'作为其最后一个字符
f	输入实数，可以用小数形式或指数形式输入
e, E, g, G	与f作用相同，e与f、g可以互相替换(大小写作用相同)

* 表 3 . 9 scanf 函数中用到的格式附加字符

字符	说 明
l	输入长整型数据(可用%ld, %lo, %lx, %lu) 以及double型数据(用%lf或%le)
h	输入短整型数据(可用%hd, %ho, %hx)
域宽	指定输入数据所占宽度(列数), 域宽应为正整数
*	本输入项在读入后不赋给相应的变量

这两个表是为了备查用的，不必死记。开始时会用比较简单形式输入数据即可。

3. 使用scanf函数时应注意的问题

(1) scanf 函数中的格式控制后面应当是**变量地址**，而不是变量名。例如，若a 和 b 为整型变量，如果写成

```
scanf("%f%f%f",a,b,c);
```

是不对的。应将“a,b,c” 改为“&a,&b,&c”。 许多初学者常犯此错误。

(2) 如果在格式控制字符串中除了格式声明以外还有其他字符，则在输入数据时在对应的位置上应输入与这些字符相同的字符。如果有

```
scanf("a=%f,b=%f,c=%f",&a,&b,&c);
```

在输入数据时，应在对应的位置上输入同样的字符。即输入

```
a=1,b=3,c=2L
```

(注意输入的内容)

如果输入

```
132
```

就错了。因为系统会把它和scanf 函数中的格式字符串逐个字符对照检查的，只是在%f 的位置上代以一个浮点数。

注意：在“a=1” 的后面输入一个逗号，它 与 scanf 函数中的“格式控制” 中的逗号对应。如果输入时不用逗号而用空格或其他字符是不对的。

(3) 在用“%c”格式声明输入字符时，空格字符和“转义字符” 中的字符都作为有效字符输入，例如：

```
scanf("%c%c%c",&c1,&c2,&c3);
```

在执行此函数时应该连续输入3个字符，中间不要有空格。如：

```
abc
```

(字符间没有空格)

若在两个字符间插入空格就不对了。如：

a bcL

系统会把第1个字符' a ' 送 给 c1; 第2个字符是空格字符' ' , 送给 c2; 第 3 个 字 符' b ' 送 给

c3。而并不是把'a'送给c1,把'b'送给c2,把'c'送给c3。

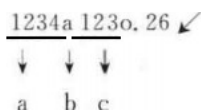
提示: 输入数值时,在两个数值之间需要插入空格(或其他分隔符),以使系统能区分两个数值。在连续输入字符时,在两个字符之间不要插入空格或其他分隔符(除非在scanf函数中的格式字符串中有普通字符,这时在输入数据时要在原位置插入这些字符),系统能区分两个字符。

(4) 在输入数值数据时,如输入空格、回车、Tab 键或遇非法字符(不属于数值的字符),认为该数据结束。例如:

```
scanf("%d%c%f",&a,&b,&c);
```

若输入

```
1234a 123o.26
```



第1个数据对应%d 格式,在输入1234之后遇字符'a',因此系统认为数值1234后已没有数字了,第1个数据应到此结束,就把1234送给变量a。把其后的字符'a'送给字符变量b,由于%c 只要求输入一个字符,系统判定该字符已输入结束,因此输入字符a 之后不需要加空格。字符'a'后面的数值应送给变量c。如果由于疏忽把1230.26错打成123o.26,由于123后面出现字母o,就认为该数值数据到此结束,将123送给变量c,后面几个字符没有被读入。

3.5.5 字符输入输出函数

除了可以用printf 函数和scanf 函数输出和输入字符外,C 函数库还提供了一些专门用于输入和输出字符的函数。它们是很容易理解和使用的。

1. 用 putchar 函数输出一个字符

想从计算机向显示器输出一个字符,可以调用系统函数库中的putchar 函数(字符输出函数)。

putchar 函数的一般形式为

```
putchar(c)
```

putchar 是 put character(给字符)的缩写,很容易记忆。C 语言的函数名大多是可以见名知义的,不必死记。putchar(c) 的作用是输出字符变量c 的值,显然输出的是一个字符。

【例3.8】先后输出BOY 三个字符。

解题思路: 定义3个字符变量,分别赋以初值'B','O','Y',然后用putchar 函数输出这3个字符变量的值。

编写程序:

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    char    a='B',b='O',c='Y';
```

```
    //定义3个字符变量并初始化
```

```
putchar(a);
putchar(b);
putchar(c);
putchar('\n');
return 0;
```

//向显示器输出字符 B
//向显示器输出字符 O
//向显示器输出字符 Y
//向显示器输出 一个换行符

运行结果：

BOY

连续输出B,O,Y 三个字符，然后换行。

从此例可以看出：用putchar 函数既可以输出能在显示器屏幕上显示的字符，也可以输出屏幕控制字符，如 putchar('\n') 的作用是输出一个换行符，使输出的当前位置移到下一行的开头。

如果把上面的程序改为以下这样，请思考输出结果。

```
#include<stdio.h>
int main()

int a=66,b=79,c=89;
putchar(a);
putchar(b);
putchar(c);
putchar('\n');
return 0;
```

//定义3个整型变量，并初始化
//向显示器输出字符B
//向显示器输出字符O
//向显示器输出字符 Y
//向显示器输出一个换行符

运行结果：

BOY

从前面的介绍已知：字符类型也属于整数类型，因此将一个字符赋给字符变量和将字符的ASCII 代码赋给字符变量作用是完全相同的(但应注意，整型数据的范围为0~127)。putchar 函数是输出字符的函数，它输出的是字符而不能输出整数。66是字符 B 的 ASCII 代码，因此，putchar(66) 输出字符B。其他类似。

说明：putchar(c) 中的 c 可以是字符常量、整型常量、字符变量或整型变量(其值在字符的 ASCII 代码范围内)。

可以用putchar 函数输出转义字符，例如：

```
putchar('\101')    (输出字符A)
```


`putchar("\n")` (括号中的\是转义字符代表单撇号，输出单撇号字符)

`putchar('\015')` (八进制数15等于十进制数13, 从附录 A 查出13是“回车”的ASCII 代码，因此输出回车，不换行，使输出的当前位置移到本行开头)

2. 用 `getchar` 函数输入一个字符

为了向计算机输入一个字符，可以调用系统函数库中的 `getchar` 函数(字符输入函数)。

getchar 函数的一般形式为

getchar()

getchar 是 get character(取得字符)的缩写, getchar 函数没有参数,它的作用是从计算机终端(一般是键盘)输入一个字符,即计算机获得一个字符。getchar 函数的值就是从输入设备得到的字符。getchar 函数只能接收一个字符。如果想输入多个字符就要用多个 getchar 函数。

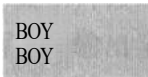
【例3.9】 从键盘输入BOY 3个字符,然后把它们输出到屏幕。

解题思路: 用 3 个 getchar 函数先后从键盘向计算机输入 BOY 3 个字符,然后用 putchar 函数输出。

编写程序:

```
#include<stdio.h>
int main()
{char a,b,c;
    a=getchar();           //定义字符变量a,b,c
    b=getchar();           //从键盘输入一个字符,送给字符变量a
    c=getchar();           //从键盘输入一个字符,送给字符变量b
    putchar(a);            //从键盘输入一个字符,送给字符变量c
    putchar(b);            //将变量a的值输出
    putchar(c);            //将变量b的值输出
    putchar("\n");         //将变量c的值输出
    return 0;              //换行
```

运行结果:



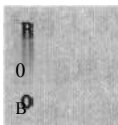
```
BOY
BOY
```

注意: 在连续输入BOY 并按 Enter 键后,字符才送到计算机中,然后输出 BOY 3 个字符。

说明: 在用键盘输入信息时,并不是在键盘上敲一个字符,该字符就立即送到计算机中去的。这些字符先暂存在键盘的缓冲器中,只有按了Enter 键才把这些字符一起输入到计算机中,然后按先后顺序分别赋给相应的变量。

如果在运行时,每输入一个字符后马上按Enter 键,会得到什么结果?

运行情况:



```
B
O
Y
```

输入字符B 后马上按 Enter，再输入字符O, 按 Enter。 立即会分两行输出B 和 O。

请思考是什么原因？

第1行输入的不是一个字符 B, 而是两个字符： B 和换行符，其中字符B 赋给了变量 a, 换行符赋给了变量b。第2行接着输入两个字符： O 和换行符，其中字符 O 赋给了变量c, 换行符没有送入任何变量。在用putchar 函数输出变量 a,b,c 的值时，就输出了字符 B, 然

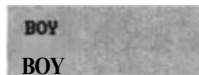
后输出换行，再输出字符O，然后执行`putchar('\n')`，换行。

注意：执行`getchar` 函数不仅可以从输入设备获得一个可显示的字符，而且可以获得在屏幕上无法显示的字符，如控制字符。

用 `getchar` 函数得到的字符可以赋给一个字符变量或整型变量，也可以不赋给任何变量，而作为表达式的一部分，在表达式中利用它的值。例如，例3.9可以改写如下：

```
#include<stdio.h>
int main()
{
    putchar(getchar());
    putchar(getchar());           //将接收到的字符输出
    putchar(getchar());           //将接收到的字符输出
    putchar('\n');                 //将接收到的字符输出
    return 0;                     //换行
}
```

运行结果：



连续输入BOY 后，按Enter 键，输出BOY，然后换行。

在连续输入BOY 并按Enter 键后，这些字符才被送到计算机中，然后按得到字符的顺序输出3个字符 BOY，最后再输出一个回车。因为第1个`getchar` 函数得到的值为' B'，因此`putchar(getchar ())` 相当于`putchar('B')`，输出' B'。第 2 个`getchar` 函数相当于`putchar('O')`，输出得到的值为' O'。第3个情况类似。

注意：不要在按B 后马上按回车键，这样就会把回车也作为一个字符输入。

也可以在`printf` 函数中输出刚接收的字符：

```
printf("%c",getchar());           // % c 是输出字符的格式声明
```

在执行此语句时，先从键盘输入一个字符，然后用输出格式符`%c` 输出该字符。

【例3.10】 改写例3.3程序，使之可以适用于任何大写字母。从键盘输入一个大写字母，在显示屏上显示对应的小写字母。

解题思路：用 `getchar` 函数从键盘读入一个大写字母，把它转换为小写字母，然后用 `putchar` 函数输出该小写字母。

编写程序：

```
#include<stdio.h>
int main()
```

```
    char  c1,c2;
    c1=getchar();
    c2=c1+32;

    putchar(c2);
    putchar('\n');

    return  0;
```

```
//从键盘读入一个大写字母，赋给字符变量c1
```

```
//求对应小写字母的ASCII 代码，放在字符变量 c2 中
//输出c2 的值，是一个字符
```

运行结果:



从键盘输入一个大写字母, 在显示屏上显示对应的小写字母。

当然, 也可以用 `printf` 函数输出。把最后两个 `putchar` 函数改用一个 `printf` 函数代替:

```
#include<stdio.h>
```

```
int main()
```

```
char cl,c2;
```

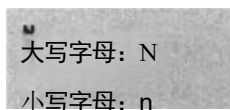
```
cl=getchar(); //从键盘读入一个大写字母, 赋给字符变量cl
```

```
c2=c1+32; //得到对应的小写字母的ASCII代码, 放在字符变量c2中
```

```
printf(" 大写字母: %c\n 小写字母: %c\n",c1,c2); // 输出cl,c2 的值
```

```
return 0;
```

运行结果:



从键盘输入一个大写字母N, 程序输出大写N 和小写n。

说明: 如果使用汉化的C 编译系统(如 Visual C++中文版), 可以在 `printf` 函数的格式字符串中包含汉字, 在输出时就能显示汉字, 以增加可读性。

思考: 可以用 `printf` 函数和 `scanf` 函数输出或输入字符, 也可以用字符输入输出函数输入或输出字符, 请比较这两个方法的特点, 在特定情况下用哪一种方法为宜。

本章结合介绍最简单的程序, 系统地介绍了编写程序的各项要素, 有了这些基础, 就可以开始编写程序了。

习 题

1. 假如我国国民生产总值的年增长率为7%, 计算10年后我国国民生产总值与现在相比增长多少百分比。计算公式为

$$p=(1+r)^n$$

r 为年增长率, n 为年数, p 为与现在相比的倍数。

2. 存款利息的计算。有1000元, 想存5年, 可按以下5种办法存:

- (1) 一次存5年期。
- (2) 先存2年期, 到期后将本息再存3年期。
- (3) 先存3年期, 到期后将本息再存2年期。

(4) 存1年期，到期后将本息再存1年期，连续存5次。

(5) 存活期存款。活期利息每一季度结算一次。

2017年的银行存款利息如下：

1年期定期存款利息为1.5%；

2年期定期存款利息为2.1%;

3年期定期存款利息为2.75%;

5年期定期存款利息为3%;

活期存款利息为0.35%(活期存款每一季度结算一次利息)。

如果 r 为年利率, n 为存款年数,则计算本息和的公式如下:

1年期本息和: $P=1000*(1+r)$;

n 年期本息和: $P=1000*(1+n*r)$;

存 n 次1年期的本息和: $P=1000*(1+r)^n$;

活期存款本息和: $P=1000 * \left(1 + \frac{r}{4}\right)^{4n}$ 。



$1000 * \left(1 + \frac{r}{4}\right)$ 是一个季度的本息和。

3. 购房从银行贷了一笔款 d ,准备每月还款额为 p ,月利率为 r ,计算多少月能还清。设 d 为300000元, p 为6000元, r 为1%。对求得的月份取小数点后一位,对第2位按四舍五入处理。

提示:计算还清月数 m 的公式如下:

$$m = \frac{\log p - \log(p - d \times r)}{\log(1 + r)}$$

可以将公式改写为

$$m = \frac{\log\left(\frac{p}{p - d \times r}\right)}{\log(1 + r)}$$

C的库函数中有求对数的函数 $\log_{10}$,是求以10为底的对数, $\log(p)$ 表示 $\log p$ 。

4. 分析下面的程序:

```
#include<stdio.h>

int main()
{char    c1,c2;
    c1=97;
    c2=98;
    printf("c1=%c,c2=%c\n",c1,c2);
    printf("c1=%d,c2=%d\n",c1,c2);
    return 0;
}
```

(1) 运行时会输出什么信息?为什么?

(2) 如果将程序第4,5行改为

$c1=197$;

$c2=198$;

运行时会输出什么信息?为什么?

(3) 如果将程序第3行改为

```
int c1,c2;
```

运行时会输出什么信息?为什么?

5. 用下面的scanf 函数输入数据,使a=3,b=7,x=8.5,y=71.82,c1='A',c2='a'。在键盘上应如何输入?

```
#include<stdio.h>
int main()
{
    int a,b;
    float x,y;
    char c1,c2;
    scanf("a=%db=%d",&a,&b);
    scanf("%f%e",&a,&y);
    scanf("%c%c",&c1,&c2);
    return 0;
```

6. 请编程序将“China”译成密码,密码规律是:用原来的字母后面第4个字母代替原来的字母。例如,字母“A”后面第4个字母是“E”,用“E”代替“A”。因此,“China”应译为“Glmre”。请编一程序,用赋初值的方法使c1,c2,c3,c4,c5 这5个变量的值分别为'C','h','i','n','a',经过运算,使c1,c2,c3,c4,c5 分别变为'G','l','m','r','e'。分别用putchar 函数和printf 函数输出这5个字符。

7. 设圆半径r=1.5,圆柱高h=3,求圆周长、圆面积、圆球表面积、圆球体积、圆柱体积。用scanf 输入数据,输出计算结果,输出时要求有文字说明,取小数点后2位数字。请编程序。

8. 编程序,用getchar 函数读入两个字符给c1 和 c2,然后分别用putchar 函数和printf 函数输出这两个字符。思考以下问题:

- (1) 变量c1 和 c2 应定义为字符型、整型还是二者皆可?
- (2) 要求输出 c1 和 c2 值的ASCII 码,应如何处理?用putchar 函数还是printf 函数?
- (3) 整型变量与字符变量是否在任何情况下都可以互相代替?如:

```
char c1,c2;
```

与

```
int c1,c2;
```

是否无条件地等价?